Instituto Tecnológico de Buenos Aires (ITBA)



25.27 - Sistemas Embebidos

Resumen

Apunte teórico-práctico basado en el contenido de clase y bibliografía oficial

Índice

1 Clase 1 4
1.1 Buses de Datos y de Direcciones 4
1.1.1 Buffers Tri-State y Lógica de Decodificación 4
1.2 GPIO (General Purpose Input/Output) 5
1.2.1 Técnica de Readback (Lectura de Salidas) 5
1.2.2 Lógica Activa: Active-High vs. Active-Low 5
1.3 Propiedades Eléctricas de los Pines Digitales 6
1.3.1 Parámetros de Configuración Dinámica de Salida 6
1.3.2 Salidas Open-Drain / Drenador Abierto 7
1.4 El Módulo GPIO en el Microcontrolador NXP Kinetis K64F 7
1.4.1 Registros de Datos CMSIS (Capa MCAL) 7
1.5 Pin Control Register (PCR) y Configuración Individual 9
1.5.1 Campos Clave del Registro `PORTx_PCRn` 9
1.5.2 Multiplexación de Pines (Campo MUX) 9
1.6 Secuencia Paso a Paso para Configurar un Pin como GPIO 9
1.6.1 Algoritmo de Inicialización Estándar 10
1.6.2 Ejemplo en Código C (Capas MCAL y Driver) 10
2 Clase 2 12
2.1 1. Bit-Banding: Operaciones Atómicas de BITS 12
2.1.1 El Problema de la Condición de Carrera (Race Condition) 12
2.1.2 La Solución: Mapeo Bit-Band 12
2.1.2.1 Fórmula de Mapeo de Direcciones 12
2.2 2. Fundamentos de Interrupciones 13

2.2.1 Conceptos Fundamentales . . . 13
2.2.2 Comparativa de Estrategias de Control . . . 13
2.2.3 Máscaras y Flags de Interrupción . . . 13
2.3 3. El Controlador de Interrupciones del K64: NVIC . . . 13
2.3.1 Mapa de Excepciones e Interrupciones . . . 14
2.3.2 Estructura de Registros del NVIC (`NVIC_Type` en CMSIS) . . . 14
2.3.3 Funciones Estándar CMSIS para el NVIC . . . 14
2.4 4. Guía Paso a Paso: Configuración de Interrupción en C . . . 14
2.4.1 Código de Ejemplo Completo (Pulsador SW3 en `PTA4` e ISR) . . . 15
3 Clase 3 . . . 17
3.1 Mecanismos Avanzados de Gestión de Prioridades en Cortex-M4 . . . 17
3.1.1 1.1. Tail-Chaining (Encadenamiento al Final) . . . 17
3.1.2 1.2. Pre-Emption (Anidamiento y Desalojo de Interrupciones) . . . 17
3.1.3 1.3. Late Arrival (Llegada Tardía) . . . 18
3.2 El Temporizador de Sistema: SysTick . . . 19
3.2.1 2.1. Propósito y Arquitectura . . . 19
3.2.2 2.2. Estructura de Registros (`SysTick_Type` en CMSIS) . . . 19
3.2.2.1 Cálculo del Valor de Recarga (`LOAD`) . . . 19
3.2.3 2.3. Código C de Inicialización y Controlador Nativo . . . 19
3.3 Patrón de Arquitectura: Despachador de Tareas Periódicas . . . 20
3.3.1 Ejemplo de Despachador Genérico en C . . . 20
3.4 Análisis Práctico de Timing y Prioridades . . . 21
4 Clase 4 . . . 22
4.1 Arquitectura de Firmware en Capas . . . 22
4.1.1 Jerarquía de Abstracción del Sistema . . . 22
4.2 Estructura Estándar de un Driver en C . . . 23
4.3 Análisis Temporal de Modelos de Ejecución . . . 23
4.3.1 1. Código Bloqueante (**Blocking Code**) . . . 23
4.3.2 2. Polling (**Consulta Continua**) . . . 24
4.3.3 3. Interrupción Periódica (**PISR / Hardware Polling**) . . . 24
4.3.4 4. Interrupción Dedicada (**ISR**) . . . 24
4.4 Integración Segura con Callbacks y Reglas de Diseño . . . 25
4.4.1 Arquitectura Correcta de Despacho de Eventos . . . 25
5 Clase 5 . . . 27
5.1 Fundamentos de Comunicación Serial vs. Paralela . . . 27
5.1.1 Comunicación Sincrónica vs. Asincrónica . . . 27
5.1.2 Buffers . . . 27
5.2 Anatomía Completa de la Trama UART (Protocolo 8N1) . . . 27
5.2.1 Eficiencia de Canal y Muestreo por Sobremuestreo . . . 28
5.3 Bitrate vs Baurate . . . 28
5.4 Capa Física, Estándares Eléctricos y Control de Flujo . . . 29
5.4.1 Control de Flujo por Software (XON / XOFF) . . . 29
5.4.2 Conversores . . . 30
5.5 Colas Circulares (Ring Buffers) e Interrupciones . . . 30
5.5.1 Código C Didáctico de una Cola Circular (`RingBuffer`) . . . 31
5.6 Mapeo de Registros y Cálculo Fraccional de Baud Rate en Kinetis K64F . . . 31
5.7 Control de Registros, FIFOs y Driver UART Completo en C . . . 32
5.7.1 Registros de Control y Estado . . . 32
5.7.2 Configuración de FIFOs de Hardware . . . 33
5.7.3 Código C Completo del Driver UART0 Orientado a Interrupciones . . . 33
6 Clase 6 . . . 34
6.1 Protocolo SPI (Serial Peripheral Interface) . . . 34
6.1.1 Arquitectura y Señales Físicas . . . 34
6.1.2 Topologías de Red . . . 35

6.2 Protocolo I2C (Inter-Integrated Circuit) . . . 35
6.2.1 Arquitectura Eléctrica y Wired-AND . . . 35
6.2.2 Formas de Onda y Reglas del Protocolo I2C . . . 35
6.2.3 Mecanismos Avanzados . . . 36
6.2.4 Driver I2C en Kinetis K64F . . . 36
7 Clase 7 . . . 39
7.1 Protocolo CAN (Controller Area Network) . . . 39
7.1.1 Arquitectura y Características Principales . . . 39
7.1.2 Capa Física y Señal Diferencial . . . 39
7.1.3 Estructura de Trama, Arbitraje y Bit Stuffing . . . 40
7.1.3.1 Trama de Datos Estándar (Standard Data Frame) . . . 40
7.1.3.2 Arbitraje No Destructivo (CSMA/CD + AMP) . . . 40
7.1.3.3 Regla de Bit Stuffing . . . 41
7.1.4 Base de Tiempo y Sincronización (Nominal Bit Time) . . . 41
7.1.4.1 División del Bit en Time Quanta (t_q . . . 41
7.1.5 Casos de Ajuste de Flanco y Resincronización (Nominal Bit Time) . . . 41
7.1.5.1 Resincronización Dinámica y SJW . . . 43
7.1.6 Filtrado por Hardware y Manejo de Errores . . . 43
7.1.6.1 Filtrado de Mensajes (Filter y Mask) . . . 43
7.1.6.2 Estados del Nodo y Manejo de Errores . . . 43

Clase 1

Buses de Datos y de Direcciones

Un microcontrolador (MCU) integra la CPU, la memoria y los periféricos en un solo chip (**System on Chip - SoC**). Para coordinar la transferencia de información entre estos bloques internos se utiliza un conjunto de conductores llamados **buses**.

Concepto de Buses de Interconexión

La CPU se comunica con la memoria RAM, Flash y los periféricos mapeados a través de tres conjuntos principales de señales:

- **Address Bus (Bus de Direcciones):** Unidireccional (proviene de la CPU). Especifica **con qué dispositivo o posición de memoria** se desea comunicar la CPU.
- **Data Bus (Bus de Datos):** Bidireccional. Transporta **el valor real** que se lee o se escribe.
- **Control Bus / Lógica de Control (R/W):** Indica el tipo de operación (Lectura: $R/w = 1$, Escritura: $R/w = 0$).

Bus / Señal	Dirección	Función Principal
Address Bus	CPU $\rightarrow$ Periférico	Selecciona la posición de memoria o registro interno a acceder.
Data Bus	CPU $\leftrightarrow$ Periférico	Transfiere la palabra de datos (8, 16 o 32 bits según la arquitectura).
Control (R/w)	CPU $\rightarrow$ Periférico	Establece si se lee (1) o si se escribe (0) sobre el bus.

Tabla 1: Resumen de los buses de interconexión interna del microcontrolador.

Buffers Tri-State y Lógica de Decodificación

Dado que múltiples periféricos (RAM, Flash, GPIO, Timers, ADC) comparten el mismo **Data Bus**, se debe evitar que más de un dispositivo intente imponer un nivel lógico simultáneamente, lo cual generaría un cortocircuito (**conflicto de bus**).

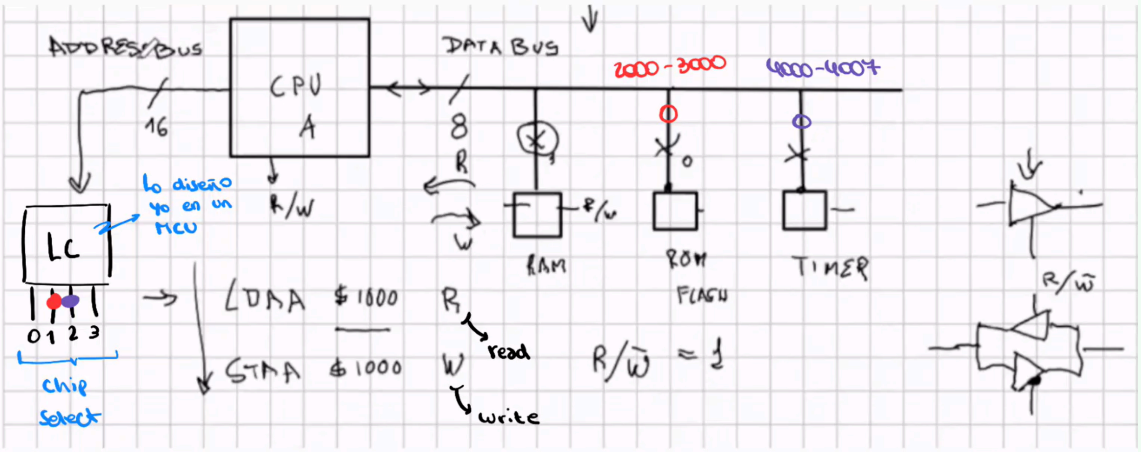
Estados Lógicos de un Buffer Tri-State:

- **Baja Impedancia:** Conectado al bus. Puede imponer nivel 1 o 0.
- **Alta Impedancia (Z):** Desconectado eléctricamente del bus. No interfiere con otros dispositivos.

Decodificador de Direcciones (Address Decoder): Analiza el **Address Bus** y activa la línea **Chip Select (CS)** únicamente del periférico correspondiente al rango de direcciones solicitado.

Ejemplo de Ciclo de Lectura / Escritura

1. **Lectura:** La CPU ejecuta la instrucción `LDAA $1000`.
   - Pone la dirección `$1000` en el **Address Bus**.
   - El **Address Decoder** activa el **Chip Select** del bloque correspondiente (ej. RAM).
   - La CPU pone $R/w = 1$.
   - El dispositivo en `$1000` pone su buffer en baja impedancia y coloca el dato en el **Data Bus**. La CPU lo guarda en el acumulador.
2. **Escritura:** La CPU ejecuta `STAA $1000`.
   - Pone la dirección `$1000` en el **Address Bus** y $R/w = 0$.
   - Coloca el valor del acumulador en el **Data Bus**. El periférico latché el dato en su registro interno.



GPIO (General Purpose Input/Output)

Un **GPIO** es un periférico modular que permite a la CPU controlar o leer pines físicos del microcontrolador como entradas o salidas digitales.

Mapeo en Memoria (Memory-Mapped I/O)

Los registros que controlan los pines del GPIO están mapeados directamente en el mapa de memoria global del microcontrolador. Escribir o leer un pin digital equivale a realizar un acceso estándar a memoria sobre una dirección específica asignada a ese registro.

Técnica de Readback (Lectura de Salidas)

El **Readback** es la capacidad del hardware de **leer el nivel lógico real** presente en un pin configurado como salida.

¿Para qué sirve el Readback?

1. **Verificación de Estado:** Confirmar si una salida físicamente cambió de estado o si existe una sobrecarga/falla en el circuito externo.
2. **Operaciones de Modificación:** Permite leer el estado actual antes de realizar un cambio lógico atómico.
3. **Diagnóstico:** Un pin de entrada configurado en el mismo nodo puede observar una salida para testear la integridad del circuito.

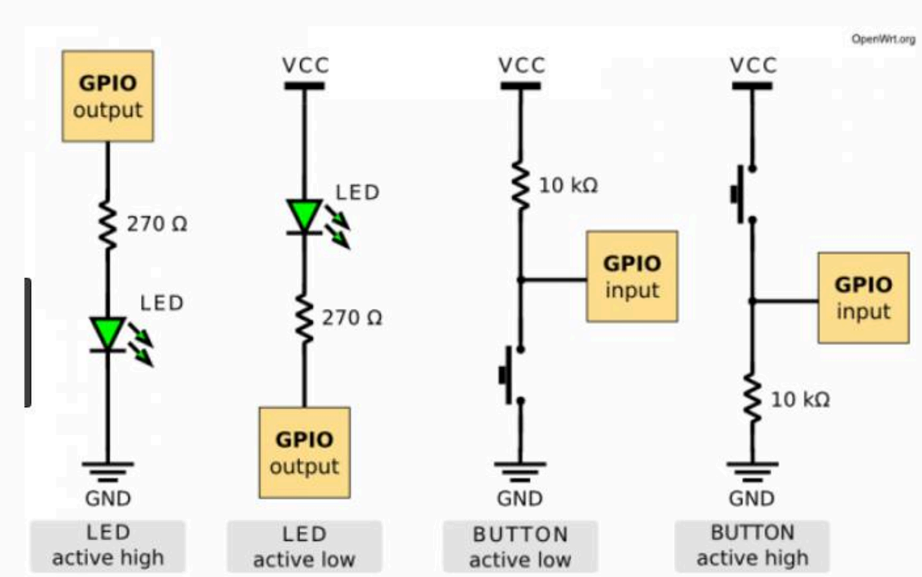
Regla de Seguridad Electrica: Nunca se deben configurar dos pines como salidas conectadas al mismo conductor con niveles opuestos (1 y 0), ya que se crea un camino directo a masa produciendo sobrecorriente y daño irreversible.

Lógica Activa: Active-High vs. Active-Low

El nivel lógico (1 ó 0) no siempre coincide con la función “activa” de un componente.

Lógica	Nivel Activo	Simbología	Ejemplo Típico
Active-High	HIGH (V_{DD})	Sin marca o `SIG`	LED encendido al enviar 1 a GND.
Active-Low	LOW (V_{SS})	Barra `SIG`, `SIG_n`, burbuja en diagramas	Botonera con Pull-Up o habilitación `LED_EN_n`.

Tabla 2: Comparación entre Lógica Activa por Alto y Lógica Activa por Bajo.

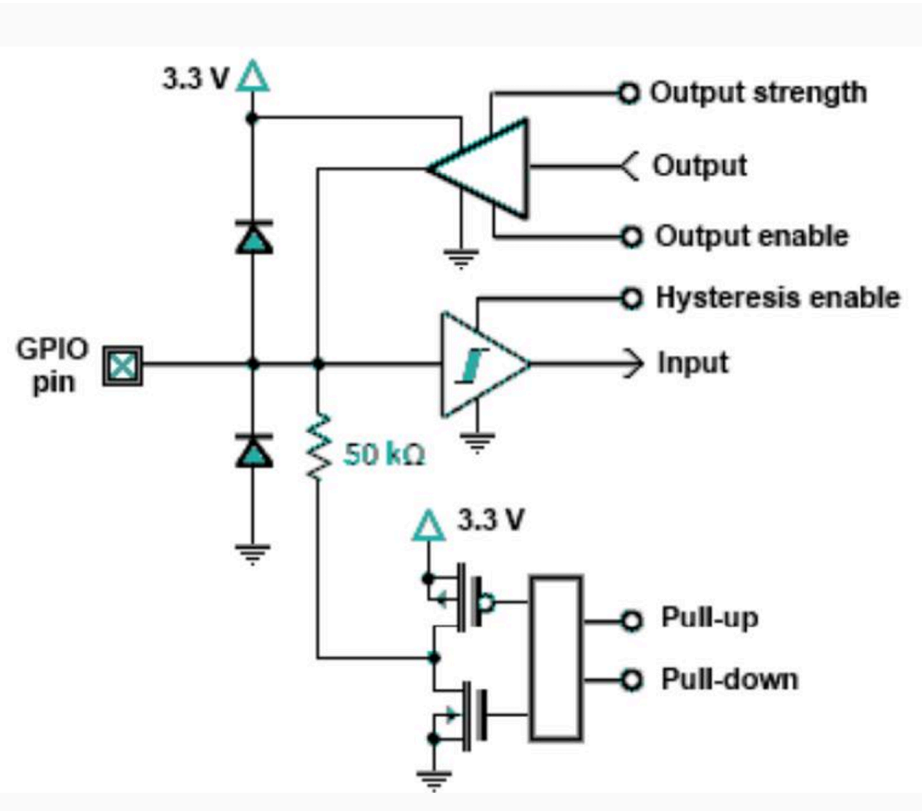


Propiedades Eléctricas de los Pines Digitales

Un pin de I/O no es un simple conductor; contiene circuitos de protección, transistores de conmutación y redes pasivas configurables por software.

Diodos de Protección (ESD/Overvoltage): Dos diodos clamping conectados entre el pin y V_{DD} / V_{SS}. Conducen cuando $V_{pin} > V_{DD} + 0.3V$ o $V_{pin} < -0.3V$. **Atención:** No deben usarse como reguladores permanentes de tensión.

Resistencias de Pull-Up / Pull-Down: Resistencias internas (típicamente $20k\Omega - 50k\Omega$) habilitables por software. Evitan que una entrada quede **flotante** (estado indeterminado sujeto a ruido electromagnético).



Parámetros de Configuración Dinámica de Salida

1. **Drive Strength (Capacidad de Corriente - DSE):**
   - **Low Drive Strength:** Limita la corriente máxima de salida (ej. $2\ \text{mA}$ a $4\ \text{mA}$). Reduce el ruido y el consumo eléctrico.
   - **High Drive Strength:** Entrega mayor corriente (ej. $18\ \text{mA}$ a $20\ \text{mA}$) para manejar cargas mayores (LEDs, buzzers) directamente sin deformar la señal.
2. **Slew Rate Control (Velocidad de Transición - SRE):**
   - **Slew Rate Enable:** Limita la velocidad de cambio $d\frac{V}{d}t$ de la transición lógico-eléctrica.
   - **Ventaja:** Atenúa picos de alta frecuencia y reduce la Interferencia Electromagnética (EMI) y la diafonía en el PCB.

Salidas Open-Drain / Drenador Abierto

En la configuración **Open-Drain**, el transistor MOSFET P superior se deshabilita. El pin solo puede conmutar entre **Masa (0)** y **Alta Impedancia (Z)**.

> **Aplicaciones de Open-Drain**
>
> - **Lógica Colector/Drenador Abierto (Wired-OR):** Permite conectar múltiples salidas al mismo bus (ej. I2C) con un único resistor de Pull-Up externo sin producir cortocircuitos.
> - **Traducción de Niveles de Tensión (Level Shifting):** Permite interfaz con circuitos de mayor voltaje (ej. controlar una línea de $5V$ alimentando el Pull-Up a $5V$ desde un microcontrolador de $3.3V$).

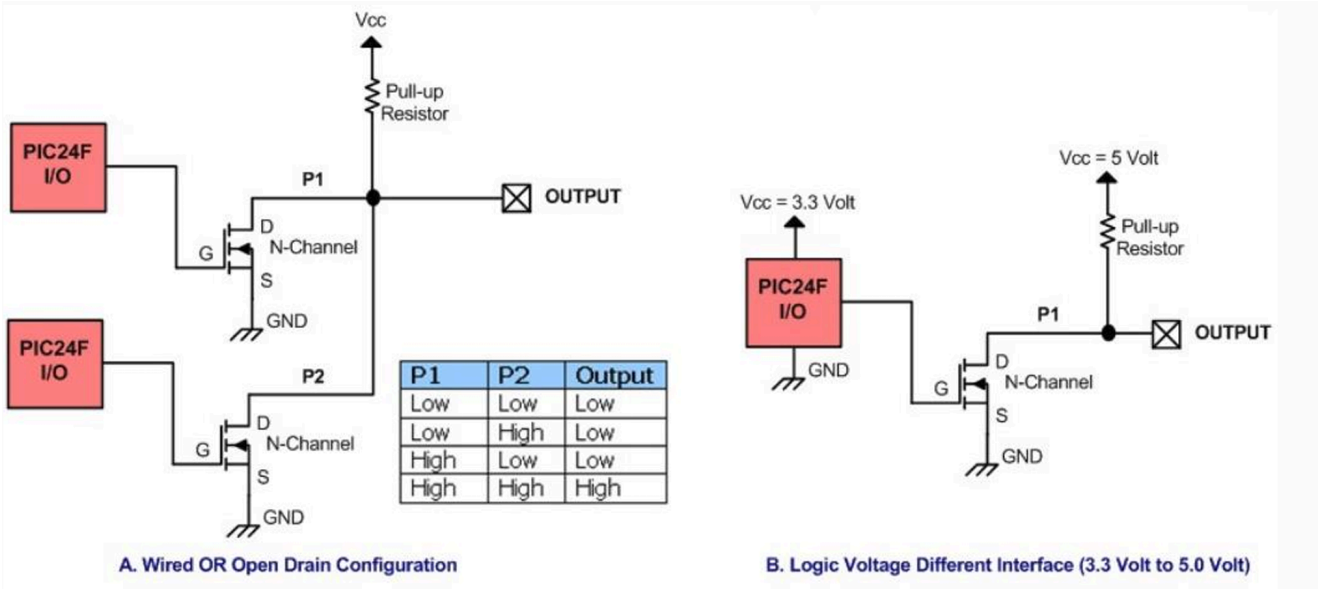


Figura 5: Implementación de lógica digital con MOSFET y elevación de la tensión de salida

El Módulo GPIO en el Microcontrolador NXP Kinetis K64F

El microcontrolador MK64FN1M0VLL12 (ARM Cortex-M4 a $120\ \text{Mhz}$) posee **5 puertos paralelos de 32 bits**: `PORTA`, `PORTB`, `PORTC`, `PORTD` y `PORTE`.

Registros de Datos CMSIS (Capa MCAL)

Para operar los datos del GPIO, el SDK provee punteros a la estructura `GPIO_Type` (`PTA`, `PTB`, `PTC`, `PTD`, `PTE`). Cada puerto cuenta con registros de 32 bits dedicados (Kinetis K64 - Reference Manual - Chapter 4):

GPIOx_PDDR — Port Data Direction Register (Offset: 0x14)

Define la dirección del pin.
- Bit en `0`: Pin configurado como **Entrada Digital (INPUT)**.

- Bit en `1`: Pin configurado como **Salida Digital (OUTPUT)**.

GPIOx_PSOR

Offset: 0x04

Port Set Output Register: Escribir un `1` en el bit n pone la salida en **HIGH (1)**. Escribir `0` no produce ningún efecto. Operación atómica sin modificar otros pines.

GPIOx_PCOR

Offset: 0x08

Port Clear Output Register: Escribir un `1` en el bit n fuerza la salida a **LOW (0)**. Escribir `0` no produce ningún efecto. Operación atómica.

GPIOx_PTOR

Offset: 0x0C

Port Toggle Output Register: Escribir un `1` en el bit n **invierte el estado** de la salida ($0 \rightarrow 1$ ó $1 \rightarrow 0$). Operación atómica.

GPIOx_PDIR

Offset: 0x10

Port Data Input Register: Registro de sólo lectura (`__I volatile const`). Retorna el valor lógico real capturado en los pines del puerto.

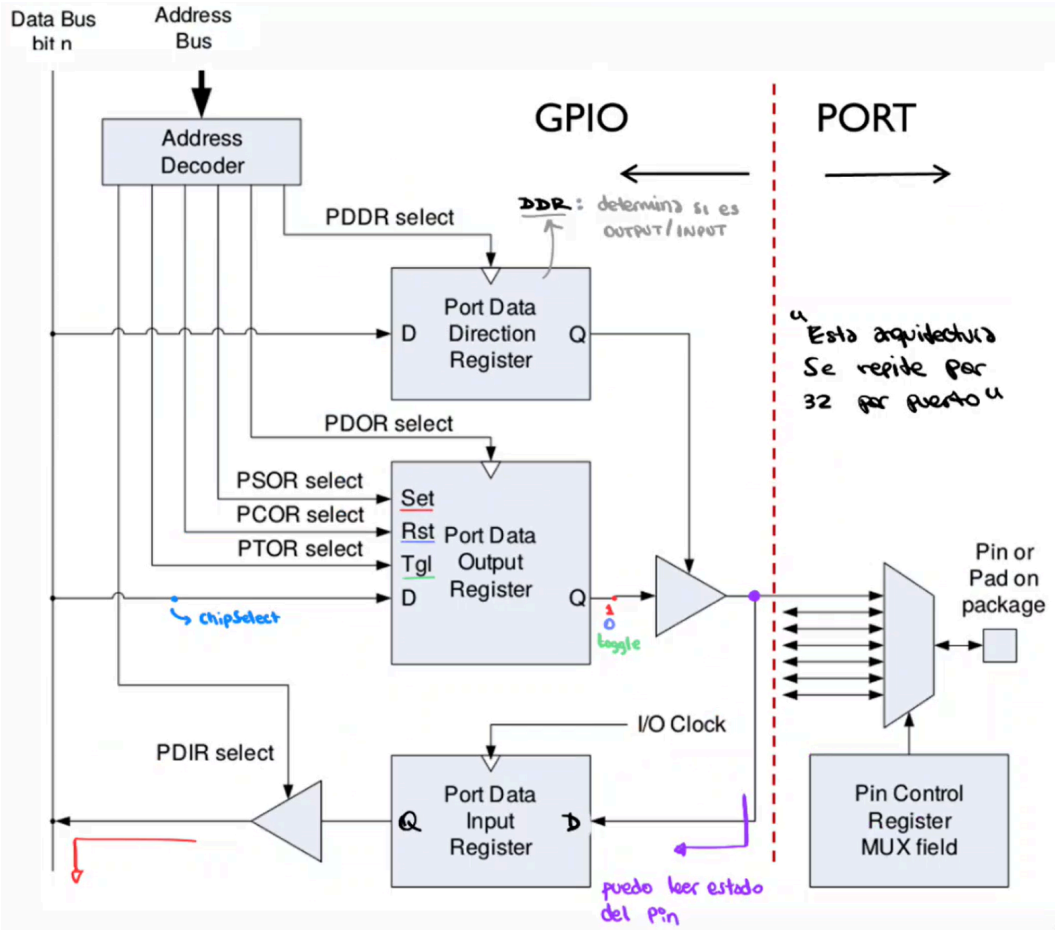


Figura 6: Registros de datos de un puerto GPIO

Ventaja de PSOR / PCOR / PTOR sobre Read-Modify-Write

En microcontroladores tradicionales de 8 bits, para cambiar un solo bit se requería leer todo el puerto, hacer una máscara `OR`/`AND` y reescribir (`Read-Modify-Write`). Esto consume múltiples ciclos de instrucción y **no**

es atómico (es susceptible a interrupciones intermedias). En el K64F, escribir en `PSOR`, `PCOR` o `PTOR` altera **exclusivamente el bit deseado en un único ciclo de reloj de bus**.

Pin Control Register (PCR) y Configuración Individual

Cada pin físico de cada puerto posee su propio registro de control individual de 32 bits llamado **`PORTX_PCRn`** (donde $X \in \{A, B, C, D, E\}$ y $n \in \{0...31\}$).

Campos Clave del Registro `PORTx_PCRn`

Campo	Bits	Descripción y Valores
ISF	24	**Interrupt Status Flag:** Se pone en 1 cuando ocurre el evento de interrupción configurado. Se limpia escribiendo un 1 (**w1c**).
IRQC	19:16	**Interrupt Configuration:** Selecciona el modo de interrupción/DMA (`0000` = Deshabilitado; `1001` = Flanco de Subida; `1010` = Flanco de Bajada; `1011` = Ambos Flancos; `1100` = Nivel Lógico 0).
LK	15	**Lock Register:** Si se escribe 1, bloquea los bits $[15:0]$ del PCR hasta el próximo Reset del microcontrolador.
MUX	10:8	**Pin Mux Control:** Selecciona qué periférico interno se conecta al pin físico. Ver tabla MUX abajo.
DSE	6	**Drive Strength Enable:** 0 = Baja capacidad de corriente; 1 = Alta capacidad de corriente.
SRE	2	**Slew Rate Enable:** 0 = Rápido (Fast); 1 = Lento/Controlado (Slow).
PE	1	**Pull Enable:** 0 = Resistencia de Pull deshabilitada; 1 = Habilitada.
PS	0	**Pull Select:** 0 = Internal **Pull-Down**; 1 = Internal **Pull-Up** (requiere `PE = 1`).

Tabla 3: Distribución de campos en el Pin Control Register (PCR).

Multiplexación de Pines (Campo MUX)

Valor MUX	Función del Pin
`000` (ALT0)	Pin Deshabilitado / Entrada Analógica (ADC/CMP)
`001` (ALT1)	**GPIO (General Purpose I/O)**
`010` (ALT2)	Periférico dedicado 1 (ej. UART, SPI)
`011` (ALT3)	Periférico dedicado 2 (ej. FTM / Timer)
`100..111`	Otras alternativas del Datasheet (ej. Trace, I2S)

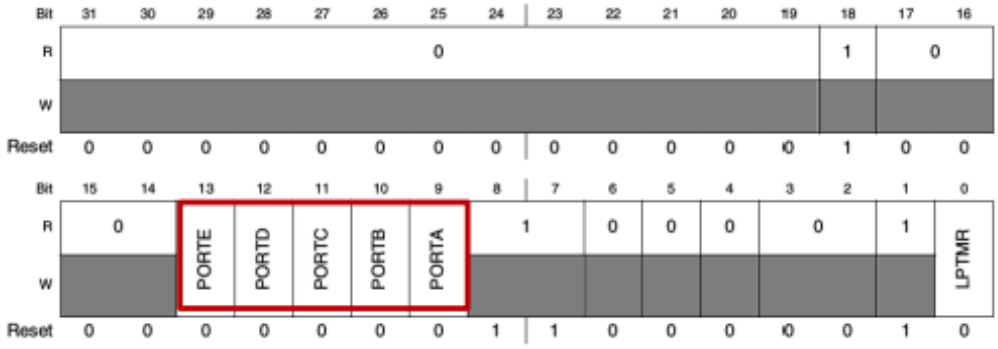
Registros Globales de PCR: Para evitar configurar los 32 pines uno por uno en código redundante:

- **`PORTx_GPCLR`:** Escribe la misma configuración en los bits $[15:0]$ del PCR de múltiples pines.
- **`PORTx_GPCHR`:** Idem para los bits $[31:16]$.

Secuencia Paso a Paso para Configurar un Pin como GPIO

REGLA DE ORO: System Clock Gating Control (SCGC)

Por defecto, tras un Reset, **los relojes de todos los puertos están deshabilitados** para minimizar el consumo. Intenta leer o escribir cualquier registro de un puerto (`PCR` o `GPIO`) sin haber habilitado primero su reloj en `SIM->SCGC5` provocará una excepción de hardware inmediata (**HardFault**).



SIM_SCGC5 field descriptions

Port X Clock Gate control

This bit controls the clock gate to the port X module

0 Clock disabled
1 Clock enabled

X= A,B,C,D,E

Algoritmo de Inicialización Estándar

1. **Habilitar el Clock del Puerto:** Setear el bit correspondiente en `SIM->SCGC5` (ej. `SIM_SCGC5_PORTB_MASK`).
2. **Configurar el Registro PCR (`PORTx_PCRn`):**
   - Asignar `MUX = 1` (`ALT1`) para operar como GPIO.
   - Configurar resistores de Pull-Up/Pull-Down, Drive Strength y Slew Rate según la necesidad eléctrica del circuito.
3. **Definir Dirección en `GPIOx_PDDR`:**
   - Setear bit en 0 para Entrada (INPUT).
   - Setear bit en 1 para Salida (OUTPUT).
4. **Operación de Lectura / Escritura:** Operar sobre los registros de datos (`PSOR`, `PCOR`, `PTOR`, `PDIR`).

Ejemplo en Código C (Capas MCAL y Driver)

El siguiente código ilustra la inicialización y uso de un LED (Salida en `PTB21`) y un Pulsador con Pull-Up (Entrada en `PTA4`):

```c
#include "MK64F12.h"
#include <stdbool.h>

void App_Init(void) {
    // 1. GATING: Habilitar relójes de los puertos A y B en la SIM. Operacion OR
    SIM->SCGC5 |= SIM_SCGC5_PORTA_MASK | SIM_SCGC5_PORTB_MASK;

    // 2. CONFIGURACIÓN DE SALIDA (LED Rojo en PTB21)
    // Limpiar PCR y seleccionar MUX = ALT1 (GPIO) + High Drive Strength
    PORTB->PCR[21] = PORT_PCR_MUX(1) | PORT_PCR_DSE_MASK;

    // Configurar dirección como Salida en PDDR
```

```
    PTB->PDDR |= (1 << 21);

    // Estado inicial: Apagar LED (Supóngase Active-Low, se envía HIGH con PSOR)
    PTB->PSOR = (1 << 21);

    // 3. CONFIGURACIÓN DE ENTRADA (Pulsador SW3 en PTA4)
    // MUX = ALT1 (GPIO) + Habilitar Pull-Up interno (PE=1, PS=1)
    PORTA->PCR[4] = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK;

    // Configurar dirección como Entrada en PDDR (bit en 0)
    PTA->PDDR &= ~(1 << 4);
}

void App_Run(void) {
    // Leer estado del pulsador desde PDIR
    bool sw3_presionado = !(PTA->PDIR & (1 << 4)); // Active-Low

    if (sw3_presionado) {
        // Si se presiona, encender LED (forzar a LOW con PCOR)
        PTB->PCOR = (1 << 21);
    } else {
        // De lo contrario, apagar LED (forzar a HIGH con PSOR)
        PTB->PSOR = (1 << 21);
    }
}
```

Clase 2

1. Bit-Banding: Operaciones Atómicas de BITS

En los microcontroladores estándar de 32 bits, la CPU **no puede modificar bits individuales directamente en memoria o en un registro**. Los accesos a memoria se realizan siempre por bytes (8 bits), half-words (16 bits) o words completos (32 bits).

El Problema de la Condición de Carrera (Race Condition) Cuando intentamos cambiar un solo bit mediante la secuencia tradicional **Read-Modify-Write (RMW)**:
1. **Read:** La CPU lee el registro completo de 32 bits a un registro interno de la CPU.
2. **Modify:** Realiza una operación lógica (`OR`, `AND`) para alterar el bit deseado.
3. **Write:** Escribe el valor modificado de regreso al registro.

> ⚠️ **Riesgo de Concurrencia con Interrupciones**
>
> Si una interrupción (ISR) ocurre justo entre el paso de **Lectura** y el de **Escritura**, la ISR puede modificar otro bit del mismo registro. Al retornar de la interrupción, el programa principal ejecutará el paso de **Escritura**, sobrescribiendo y perdiendo completamente el cambio realizado por la ISR.

La Solución: Mapeo Bit-Band La arquitectura ARM Cortex-M4 introduce el mecanismo de Bit-Banding, el cual convierte un acceso a un **bit individual** en una operación de escritura o lectura **atómica** (indivisible e ininterrumpible) mediante el uso de un alias de memoria.

> 💡 **Región Bit-Band**
>
> - Corresponde a la región de memoria real (SRAM o Periféricos).
> - Cada bit en esta región se mapea a una palabra de 32 bits en la región alias.
> - Tamaño: 1 MB de memoria real.

> 💡 **Región Alias Bit-Band**
>
> - Dirección especial de memoria de 32 MB.
> - Escribir `0x01` o `0x00` en una dirección alias modifica solamente el bit objetivo de manera atómica por hardware.

2.1.2.1 Fórmula de Mapeo de Direcciones

Para calcular la dirección en la región alias correspondiente a un bit específico de un byte/registro:

$$\text{Alias_Addr} = \text{Alias_Base} + (\text{Byte_Offset} \times 32) + (\text{Bit_Number} \times 4)$$

Zona de Memoria	Dirección Base Real (Bit-Band)	Dirección Base Alias
SRAM (RAM)	`0x2000_0000`	`0x2200_0000`
Periféricos (GPIO, TIM, etc.)	`0x4000_0000`	`0x4200_0000`

> 🛠️ **Lectura / Escritura en Alias**
>
> - Escritura: Escribir cualquier número con LSB=1 (como `0x01`) en la dirección alias pone el bit real en `1`. Escribir un número con LSB=0 (como `0x00`) pone el bit real en `0`.
> - Lectura: Leer la dirección alias retorna `0x01` si el bit real está en `1`, y `0x00` si está en `0`.

2. Fundamentos de Interrupciones

Una interrupción es un evento generado por el hardware que suspende temporalmente la ejecución secuencial del programa principal (**Main Loop**) para atender una tarea urgente mediante una rutina llamada ISR (**Interrupt Service Routine**). Una vez finalizada la ISR, el procesador retoma la ejecución del programa exactamente en la instrucción donde fue suspendido.

Conceptos Fundamentales

- IRQ (Interrupt Request): Solicitud asincrónica de interrupción enviada por un periférico al procesador.
- ISR (Interrupt Service Routine / Handler): Función ejecutada en respuesta a una IRQ específica.
- IVT (Interrupt Vector Table): Tabla ubicada al inicio de la memoria que almacena punteros a función (**vectores**) con las direcciones de inicio de cada ISR.

Comparativa de Estrategias de Control

Mecanismo	Funcionamiento	Ventajas	Desventajas
Polling (Consulta)	El bucle principal consulta continuamente el estado del periférico en un `while(1)`.	Fácil de programar, sin sobrecosto de contexto.	Desperdicia tiempo de CPU y energía; no determinístico.
Interrupción Periódica (Tick)	Un timer (SysTick) interrumpe a intervalos regulares para muestrear eventos (p. ej. botones).	Consumo controlado, excelente para debouncing de pulsadores.	Retardo de respuesta acotado por el período del tick.
Interrupción Dedicada	El evento de hardware exterior (flanco en pin) dispara la ISR en tiempo real.	Respuesta inmediata (mínima latencia), ultra eficiente.	Puede saturar la CPU si ocurren eventos indeseados a muy alta frecuencia.

Máscaras y Flags de Interrupción Para que una interrupción sea atendida, se requieren dos llaves en serie:

💡
1. Máscara Global

- Controlada por el procesador (**Global Interrupt Enable** / bit `I` en `PRIMASK`).
- Permite habilitar o deshabilitar todas las interrupciones enmascarables de forma centralizada (funciones `hw_EnableInterrupts()` / `hw_DisableInterrupts()`).

💡
2. Máscaras Individuales

- Nivel Periférico: Bit `IRQC` en el `PCR` del pin.
- Nivel NVIC: Registros `ISER` / `ICER` del controlador de interrupciones del ARM Cortex-M4.

⚠️ **Flags de Interrupción y Lectura/Limpieza (w1c)**

Cuando ocurre el evento, el hardware activa un bit de estado llamado Interrupt Status Flag (ej. en `ISFR`).

Regla de Oro: En la mayoría de las ISRs de periféricos (como los puertos GPIO), es obligatorio limpiar el flag dentro de la rutina escribiendo un `1` (Write-1-to-Clear / `w1c`). Si no se limpia el flag, al salir de la ISR el procesador verá el flag encendido y volverá a entrar a la interrupción en un bucle infinito.

3. El Controlador de Interrupciones del K64: NVIC

El NVIC (**Nested Vectored Interrupt Controller**) es el módulo integrado en el núcleo ARM Cortex-M4 que administra todas las excepciones e interrupciones del microcontrolador.

💡 Características del NVIC

1. Vectoreado: Determina la dirección de la ISR directamente mediante la tabla IVT.
2. Anidado (Nested): Si se dispara una interrupción de **mayor prioridad** mientras se ejecuta otra ISR de menor prioridad, la primera es suspendida para atender a la más urgente.
3. Tail-Chaining: Optimización por hardware que reduce el tiempo de cambio entre dos interrupciones consecutivas a solo 6 ciclos de reloj (evita desapilar y volver a apilar registros innecesariamente).

Mapa de Excepciones e Interrupciones Las excepciones se dividen en internas (del núcleo ARM Cortex) y externas (periféricos del fabricante NXP):

Nº Excepción	Nombre	Prioridad	Descripción
1	Reset	−3 (Máxima)	Reinicio del sistema por hardware o software.
2	NMI	−2	Interrupción No Enmascarable (urgencias graves).
3	HardFault	−1	Falla grave de hardware (ej. acceso no permitido a memoria).
15	SysTick	Configurable	Timer de sistema periódico propio del núcleo ARM.
16 (IRQ0)	DMA / UART / PORTA…	Configurable	Interrupciones externas de periféricos del K64F.

Estructura de Registros del NVIC (`NVIC_Type` en CMSIS) El NVIC se programa mediante una estructura de registros mapeados en memoria:

⚙️ Registros Principales del NVIC

- `ISER[8]` (Interrupt Set-Enable): Escribir `1` habilita la línea de interrupción individual.
- `ICER[8]` (Interrupt Clear-Enable): Escribir `1` deshabilita la línea de interrupción.
- `ISPR[8]` (Interrupt Set-Pending): Fuerza la interrupción a estado pendiente por software.
- `ICPR[8]` (Interrupt Clear-Pending): Limpia el estado pendiente.
- `IABR[8]` (Interrupt Active Bit): Indica si una ISR está actualmente en ejecución.
- `IP[240]` (Interrupt Priority): Array de bytes para configurar la prioridad. En el K64F se utilizan solo los 4 bits más significativos de cada byte ($2^4 = 16$ niveles de prioridad, donde 0 es la máxima prioridad y 15 la mínima).

Funciones Estándar CMSIS para el NVIC

```
// Habilitar / Deshabilitar interrupción de un periférico
NVIC_EnableIRQ(PORTA_IRQn);
NVIC_DisableIRQ(PORTA_IRQn);

// Configurar prioridad (IRQn_Type IRQn, uint32_t priority [0..15])
NVIC_SetPriority(PORTA_IRQn, 5);

// Limpiar estado pendiente
NVIC_ClearPendingIRQ(PORTA_IRQn);
```

4. Guía Paso a Paso: Configuración de Interrupción en C

Para configurar e implementar una interrupción de GPIO externa (por ejemplo, en el pulsador SW3 conectado a `PTA4`), se debe seguir estrictamente la siguiente secuencia de 5 pasos:

🛠️ Secuencia de Inicialización de una Interrupción (5 Pasos Globales)

1. Deshabilitar Interrupciones Globales: Evita que entren interrupciones mientras se configura el microcontrolador (`hw_DisableInterrupts()`).
2. Clock Gating: Activar el reloj del puerto correspondiente (`SIM->SCGC5 |= SIM_SCGC5_PORTA_MASK`).
3. Configurar Pin Control Register (PCR): Configurar MUX como GPIO (ALT1), resistencias de Pull-Up/Down y el modo de disparo por flanco en `IRQC`.
4. Habilitar IRQ en el NVIC: Activar la atención de interrupción del puerto en el controlador del núcleo (`NVIC_EnableIRQ(PORTA_IRQn)`).
5. Habilitar Interrupciones Globales: Permitir la entrada de interrupciones al procesador (`hw_EnableInterrupts()`).

Código de Ejemplo Completo (Pulsador SW3 en `PTA4` e ISR)

```
#include "MK64F12.h"

// 1. Definición de la Rutina de Servicio de Interrupción (ISR)
void PORTA_IRQHandler(void) {
    // A) Verificar e identificar el pin que interrumpió
    if (PORTA->ISFR & (1 << 4)) {

        // B) LIMPIAR EL FLAG DE INTERRUPCIÓN (Write 1 to Clear)
        PORTA->ISFR = (1 << 4);

        // C) Ejecutar la acción deseada (ejemplo: alternar LED)
        PTB->PTOR = (1 << 21); // Toggle LED Azul en PTB21
    }
}

// 2. Función de Inicialización del Sistema
void App_Init(void) {
    // Paso 1: Deshabilitar interrupciones globales durante la configuración
    hw_DisableInterrupts();

    // Paso 2: Activar Clock Gating para los puertos A y B
    SIM->SCGC5 |= SIM_SCGC5_PORTA_MASK | SIM_SCGC5_PORTB_MASK;

    // Configurar LED Azul en PTB21 como salida
    PORTB->PCR[21] = PORT_PCR_MUX(1); // GPIO (ALT1)
    PTB->PDDR |= (1 << 21);           // Dirección: Salida

    // Paso 3: Configurar el Pin Control Register (PCR) del Pulsador PTA4
    PORTA->PCR[4] = 0x0;                                    // Limpiar registro
    PORTA->PCR[4] |= PORT_PCR_MUX(1);                        // MUX = ALT1 (GPIO)
    PORTA->PCR[4] |= PORT_PCR_PE(1) | PORT_PCR_PS(1);        // Habilitar Pull-Up interno
    PORTA->PCR[4] |= PORT_PCR_IRQC(0x09);                    // IRQC = 0x09 (Flanco de Subida /
Interrupt on Rising Edge)

    // Configurar PTA4 como entrada
    PTA->PDDR &= ~(1 << 4);

    // Paso 4: Configurar prioridad y habilitar la IRQ del Puerto A en el NVIC
    NVIC_SetPriority(PORTA_IRQn, 3); // Prioridad 3
    NVIC_EnableIRQ(PORTA_IRQn);      // Habilitar en NVIC

    // Paso 5: Re-habilitar interrupciones globales
    hw_EnableInterrupts();
}
```

💡 Resumen Visual de Configuración `IRQC` en PCR

Los bits [19:16] del registro `PORTx_PCRn` determinan el modo de generación de evento de interrupción/DMA:

- `0000` (0x0): Interrupción deshabilitada.
- `1000` (0x8): Interrupción lógica `0` (por nivel).
- `1001` (0x9): Interrupción por Flanco de Subida (Rising Edge).
- `1010` (0x10 / 0xA): Interrupción por Flanco de Bajada (Falling Edge).
- `1011` (0x11 / 0xB): Interrupción por Ambos Flancos (Either Edge).
- `1100` (0x12 / 0xC): Interrupción lógica `1` (por nivel).

Clase 3

Mecanismos Avanzados de Gestión de Prioridades en Cortex-M4

La arquitectura **ARM Cortex-M4** incluye optimizaciones por hardware diseñadas para minimizar la latencia de interrupción y el sobrecosto (**overhead**) del apilado y desapilado de registros en la memoria Stack.

1.1. Tail-Chaining (Encadenamiento al Final) En procesadores convencionales, al finalizar la ejecución de una ISR, la CPU realiza la restauración completa de registros (**POP**) hacia el programa principal, para luego realizar inmediatamente un guardado de registros (**PUSH**) si existe otra interrupción pendiente.

💡 Mecanismo de Tail-Chaining

- **Funcionamiento:** Si al terminar una ISR existe otra interrupción pendiente con la prioridad necesaria para ser atendida, el hardware **omite la secuencia completa de POP y PUSH**.
- **Ahorro de Ciclos:** En lugar de gastar más de 20 ciclos restaurando y volviendo a salvar el contexto, la CPU salta directamente a la segunda ISR realizando una transición administrativa de solo **6 ciclos de reloj**.
- **Beneficio:** Reduce drásticamente la latencia para atender interrupciones consecutivas acumuladas.

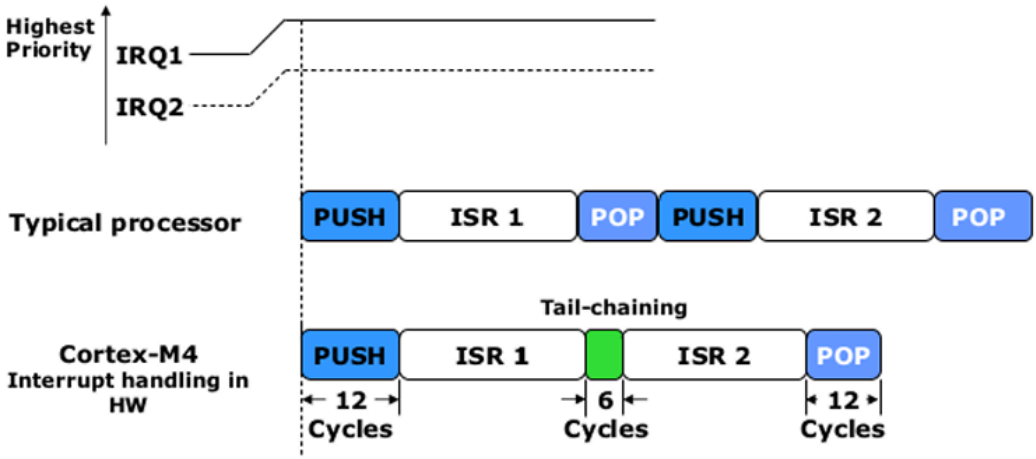


Figura 8: Interrupt Latency - Tail Chaining

1.2. Pre-Emption (Anidamiento y Desalojo de Interrupciones) El NVIC permite el anidamiento de interrupciones basado en niveles de prioridad.

- **Regla de Desalojo:** Una interrupción entrante solo desaloja (**preempts**) a la ISR que se está ejecutando actualmente si su **nivel de prioridad es strictly mayor** (es decir, posee un número numérico menor en la configuración de prioridad).
- **Prioridad Igual o Menor:** Si la interrupción entrante tiene prioridad igual o más baja que la activa, permanece retenida en estado pendiente (**Pending**) hasta que finalice la ISR en curso, momento en el cual se atiende mediante **Tail-Chaining**.

⚠️ Guardado Automático de Contexto

Cuando ocurre el desalojo por mayor prioridad, el hardware apila automáticamente el bloque de registros básicos en la pila (**MSP/PSP**) para permitir la ejecución de la nueva ISR y reanudar la ISR anterior al finalizar la más urgente.

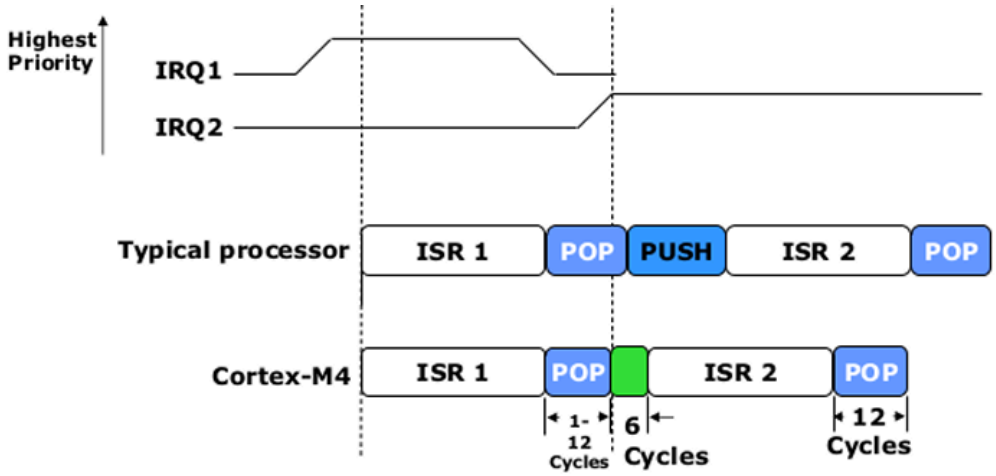


Figura 9: Interrupt Latency - Pre-Emption

1.3. Late Arrival (Llegada Tardía) Ocurre cuando una interrupción de **alta prioridad** se dispara justo mientras el procesador está realizando la fase de apilado automático de registros (**PUSH**) para una interrupción previa de **menor prioridad**.

> 💡 **Optimizador de Llegada Tardía**
>
> 1. El procesador **no interrumpe ni descarta** el proceso de apilado en curso, ya que la información del contexto que debe guardarse en el Stack es idéntica independientemente de qué ISR se ejecute primero.
> 2. En paralelo a la finalización del apilado, la CPU cambia el vector buscado en la tabla IVT y **salta a ejecutar primero la ISR de mayor prioridad** (la que llegó tarde).
> 3. Al terminar la ISR de alta prioridad, el procesador atiende a la de menor prioridad mediante **Tail-Chaining** (6 ciclos).

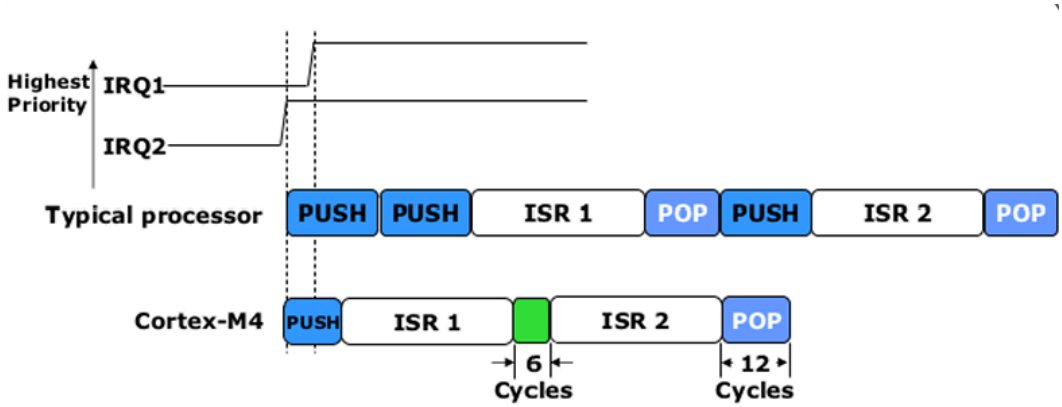


Figura 10: Interrupt Latency - Late Arrival

Mecanismo de HW	Condición de Disparo	Impacto en Latencia / Ciclos
Tail-Chaining	ISR finaliza y hay otra interrupción pendiente.	Transición directa en **6 ciclos** (omite POP + PUSH completo).
Pre-Emption	Entra IRQ con prioridad estrictamente mayor.	Desaloja ISR actual; se anida mediante nuevo PUSH de contexto.
Late Arrival	Entra IRQ de alta prioridad durante el PUSH de una menor.	Aprovecha el PUSH en curso y redirige el salto a la ISR más urgente.

El Temporizador de Sistema: SysTick

El **SysTick** (**System Tick Timer**) es un temporizador regresivo de 24 bits integrado internamente dentro del núcleo ARM Cortex-M4 (Excepción N° 15 de la tabla IVT).

2.1. Propósito y Arquitectura

- **Heartbeat del Sistema:** Proporciona una base de tiempo fija y confiable (típicamente a **1 kHz / 1 ms**) utilizada por el sistema operativo o scheduler para medir retrasos, marcas de tiempo y coordinar tareas periódicas.
- **Contador Regresivo de 24 Bits:** Módulo decrementador independiente que cuenta desde el valor cargado en `LOAD` hasta `0`.
- **Excepción del Núcleo:** Al llegar a cero, se activa el flag `COUNTFLAG` y, si está habilitado, dispara la excepción nativa `SysTick_Handler()`.

> ⚠️ **Limpieza Automática de Flag por Hardware**
>
> A diferencia de los periféricos externos de los puertos (como `PORTA` o `PORTB`), **la ISR del SysTick NO requiere limpiar un flag mediante software (w1c)**. El hardware borra automáticamente la solicitud de interrupción al ingresar a la rutina `SysTick_Handler()`.

2.2. Estructura de Registros (`SysTick_Type` en CMSIS)

El módulo se programa mediante cuatro registros principales agrupados en la estructura `SysTick`:

> ⚙️ **Registros del Módulo SysTick**
>
> - **`SysTick->CTRL` (Control and Status Register):**
> - Bit 0 (`ENABLE`): Activa o pausa el contador (1 = Encendido).
> - Bit 1 (`TICKINT`): Habilita la excepción de interrupción (1 = Genera IRQ al llegar a 0).
> - Bit 2 (`CLKSOURCE`): Selecciona la fuente de reloj (1 = Core Clock / 100 MHz; 0 = Clock externo).
> - Bit 16 (`COUNTFLAG`): Retorna 1 si el contador llegó a 0 desde la última lectura.
> - **`SysTick->LOAD` (Reload Value Register):** Registro de 24 bits (valor máx: $2^{24} - 1$). Define el valor inicial cargado al llegar a cero.
> - **`SysTick->VAL` (Current Value Register):** Valor actual del contador de 24 bits. Escribir cualquier valor borra el contador y limpia `COUNTFLAG`.
> - **`SysTick->CALIB` (Calibration Value Register):** Registro de lectura con la calibración de fábrica para 10 ms.

3.2.2.1 Cálculo del Valor de Recarga (`LOAD`)

Para obtener un intervalo de tiempo T con una frecuencia de reloj f_{core}:

$$\text{LOAD} = (f_{\text{core}} \times T) - 1$$

> 🛠️ **Ejemplos de Cálculo de Reload para K64F ($f_{\text{core}} = 100$ MHz)**
>
> - **Para Tick de 1 ms (1 kHz):**
>
> $$\text{LOAD} = (100.000.000\ \text{Hz} \times 0,001\ \text{s}) - 1 = 100.000 - 1 = 99.999\ \text{0x1869F}$$
>
> - **Para Tick de 125 ms (8 Hz):**
>
> $$\text{LOAD} = (100.000.000\ \text{Hz} \times 0,125\ \text{s}) - 1 = 12.500.000 - 1 = 12.499.999\ \text{0xBEBC1F}$$

2.3. Código C de Inicialización y Controlador Nativo

```c
#include "MK64F12.h"

#define SYSTICK_CORE_HZ  100000000UL // 100 MHz
```

```c
#define SYSTICK_FREQ_HZ  1000U       // 1 kHz (1 ms)

// 1. Inicialización del SysTick a 1 kHz
void SysTick_Init(void) {
    // Deshabilitar el temporizador antes de configurar
    SysTick->CTRL = 0x00;

    // Configurar valor de recarga para 1 ms
    SysTick->LOAD = (SYSTICK_CORE_HZ / SYSTICK_FREQ_HZ) - 1UL;

    // Limpiar el contador actual
    SysTick->VAL = 0x00;

    // Seleccionar Core Clock, habilitar interrupción y activar SysTick
    SysTick->CTRL = SysTick_CTRL_CLKSOURCE_Msk |
                    SysTick_CTRL_TICKINT_Msk   |
                    SysTick_CTRL_ENABLE_Msk;
}

// 2. Manejador de la Interrupción (ISR nativa del núcleo)
void SysTick_Handler(void) {
    // No requiere limpiar flags por software.
    // Tareas periódicas o incremento de contador de ms:
    milisegundos_transcurridos++;
}
```

Patrón de Arquitectura: Despachador de Tareas Periódicas

Para evitar configurar múltiples timers de hardware para cada tarea ligera, el SysTick actúa como una **base de tiempo compartida** mediante el registro de callbacks (**Scheduler** / Fachada HAL).

> 💡 **Ventajas del Despachador Periódico (SysTick Multi-Callback)**
>
> 1. **Consolidación:** Múltiples módulos de software (refresco de display multiplexado, lectura de encoder con antirrebote, temporizadores de software) se enganchan al mismo tick de 1 kHz.
> 2. **Desacoplamiento:** La ISR del SysTick no conoce la lógica interna de los drivers; simplemente recorre un vector de punteros a función y los ejecuta en orden secuencial.

Ejemplo de Despachador Genérico en C

```c
#define MAX_CALLBACKS 4

typedef void (*systick_callback_t)(void);

static systick_callback_t callbacks_list[MAX_CALLBACKS] = {0};
static uint8_t callback_count = 0;

// Registrar una tarea periódica
bool SysTick_AddCallback(systick_callback_t callback) {
    if (callback_count < MAX_CALLBACKS && callback != NULL) {
        callbacks_list[callback_count++] = callback;
        return true;
    }
    return false;
}

// ISR del SysTick: desparrama la ejecución a todos los callbacks
void SysTick_Handler(void) {
    for (uint8_t i = 0; i < callback_count; i++) {
```

```
        if (callbacks_list[i] != NULL) {
            callbacks_list[i](); // Ejecuta HAL_Refresh_Task, actualizarEncoder, etc.
        }
    }
}
```

Análisis Práctico de Timing y Prioridades

En sistemas embebidos reales con múltiples interrupciones concurrentes, asignar la misma prioridad a todas las ISRs puede provocar la pérdida de eventos críticos.

🛠️ Caso de Estudio: Lector de Banda Magnética vs. Refresco de Display

- **Conflicto:** La ISR de refresco de display (que hace **bit-banging** por shift-registers en SysTick) es la tarea más larga del sistema.
- **Problema:** Si el SysTick tuviera la misma prioridad que la interrupción de puerto `PORTB` del Lector de Tarjetas (ambos en prioridad 0 por defecto), la ejecución del SysTick retrasaría la captura del flanco del reloj de la tarjeta, perdiendo bits del **swipe**.
- **Solución:** Ajustar las prioridades mediante el NVIC bajándole la prioridad al SysTick:

```
// Asignación de Prioridades en App_Init()
void App_Init(void) {
    HAL_Scheduler_GlobalIRQDisable();

    // Configurar SysTick con prioridad 1 (más baja que la prioridad 0 por defecto de PORTB)
    NVIC_SetPriority(SysTick_IRQn, 1);

    // Inicializar el lector de tarjetas (queda en prioridad 0 por defecto)
    inicializarLector();

    HAL_Scheduler_GlobalIRQEnable();
}
```

💡 Verificación con Osciloscopio (Pin de Testeo TP)

Mediante el uso de un pin de prueba GPIO (`PIN_TEST_ISR`), se conmuta a nivel `HIGH` al ingresar a una ISR y a `LOW` al salir. Un osciloscopio permite medir con precisión absoluta:

- **Duración de la ISR (T_{exec}):** Porcentaje del período del tick consumido por el firmware.
- **Latencia de Interrupción:** Tiempo exacto entre el flanco físico del periférico externo y la entrada efectiva del procesador a la rutina de servicio.

Clase 4

Arquitectura de Firmware en Capas

En el desarrollo de sistemas embebidos profesionales, la complejidad crece rápidamente al integrar múltiples periféricos (displays multiplexados, teclados, sensores, comunicaciones). Para evitar que el código se vuelva inmanejable ("código espagueti") y garantizar su mantenibilidad, portabilidad y testeabilidad, se utiliza una **arquitectura modular estructurada en capas de abstracción bien definidas**.

> 💡 **Principio de Diseño en Capas**
>
> Cada capa del sistema satisface un rol específico y se comunica **exclusivamente con la capa inmediata superior o inferior** mediante interfaces estándar. La regla fundamental es el **aislamiento de hardware**: la capa de aplicación debe desconocer por completo qué microcontrolador o circuito integrado físico está ejecutando el código.

Jerarquía de Abstracción del Sistema

Capa	Responsabilidad y Función	Impacto ante Cambios
Aplicación (APP)	Contiene la lógica de negocio (FSM, reglas del producto). Es independiente del tiempo real y **nunca accede directamente a registros**.	Totalmente portable. No cambia si se reemplaza el procesador o los componentes externos.
Hardware Abstraction Layer (HAL)	Abstrae los componentes de hardware externos (display de 7 segmentos, shift-registers 74HC595, lector de tarjetas, encoder).	Si cambia un componente exterior (ej. nuevo display), **solo se modifica esta capa**.
Microcontroller Abstraction Layer (MCAL)	Librería de más bajo nivel. Modifica los registros del microcontrolador (PORT, GPIO, SysTick, ADC, PIT, UART).	Si se cambia de microcontrolador (ej. K64F → STM32), **solo se modifica esta capa**.
Complex Drivers	Módulos especiales que unifican MCAL y HAL para aplicaciones de tiempo crítico estricto o máxima velocidad.	Usados solo cuando el paso por capas intermedias introduce latencia inaceptable.

> ⚠️ **Regla Inviolable de Arquitectura**
>
> **La Capa de Aplicación (APP) jamás debe incluir `MK64F12.h` ni acceder a registros como `PORTB->PCR` o `PTB->PSOR`.** Toda interacción con el mundo físico debe realizarse a través de las funciones de servicio de la capa HAL/MCAL.

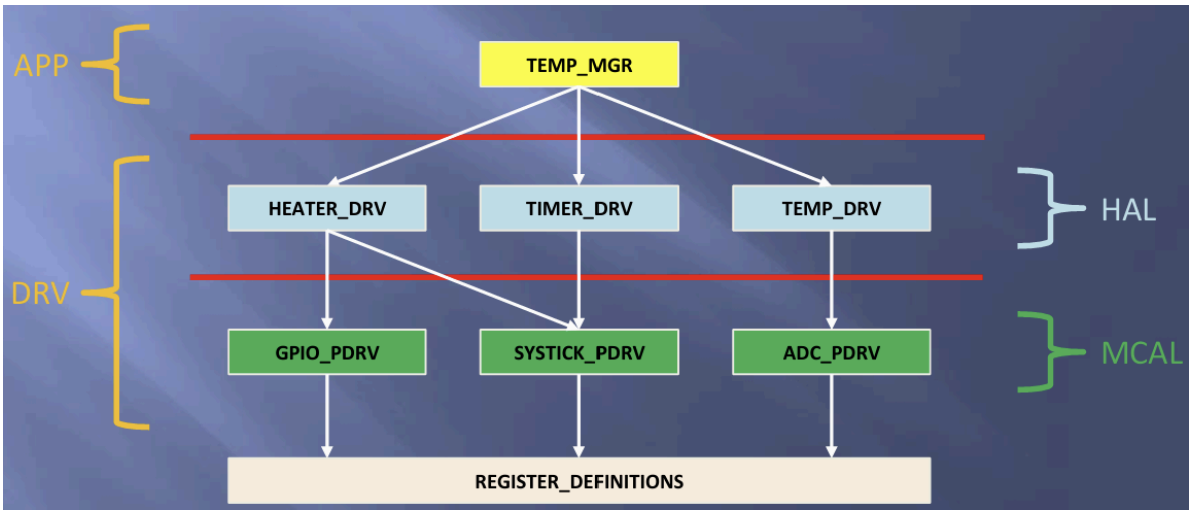


Figura 11: Ejemplo de Arquitectura de Firmware: Controlador de temperatura

Estructura Estándar de un Driver en C

Todos los drivers de periféricos (DRV) deben seguir una estructura interna uniforme dividida en 4 partes funcionales:

1. Inicialización (`DRV_Init`)

- Configura los registros iniciales del periférico e inicializa variables locales en RAM.
- Se ejecuta **una sola vez** al arrancar el programa (invocada desde `App_Init`).

2. Servicios de API (`DRV_SrvN`)

- Funciones públicas de interfaz con la capa superior (ej. `HAL_Display_SetText`).
- Invocables con cualquier frecuencia desde la APP. Ocultan los registros.

3. Rutina Periódica (`DRV_PISR`)

- Tarea ejecutada a frecuencia fija (ej. cada 1 ms vía callback del SysTick).
- Diseñada para **eventos lentos** (antirrebote de botones, multiplexado de display).

4. Interrupción Dedicada (`DRV_ISR`)

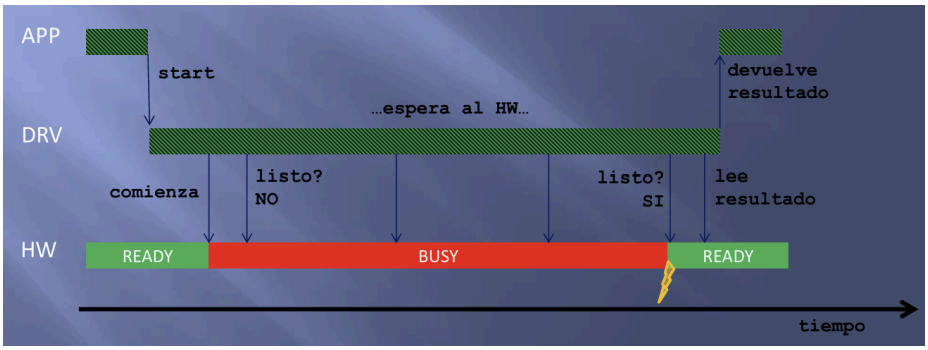
- Atiende eventos físicos instantáneos disparados por hardware.
- Diseñada para **eventos rápidos / críticos** (flancos de reloj de banda magnética).

Análisis Temporal de Modelos de Ejecución

Para gestionar la interacción entre la Aplicación, el Driver y el Hardware, existen 4 estrategias temporales con eficiencias drásticamente distintas:

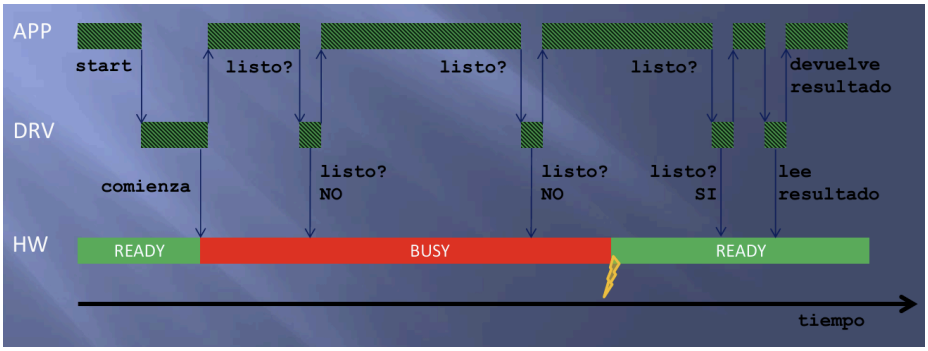
1. Código Bloqueante (Blocking Code) La Aplicación llama al Driver para iniciar una tarea y el Driver congela la CPU en un bucle cerrado hasta que el Hardware responda.

- **Rendimiento: 0% de eficiencia de CPU.** Desperdicia millones de ciclos de reloj esperando eventos externos.
- **Uso:** Totalmente **prohibido** en sistemas profesionales y de tiempo real.



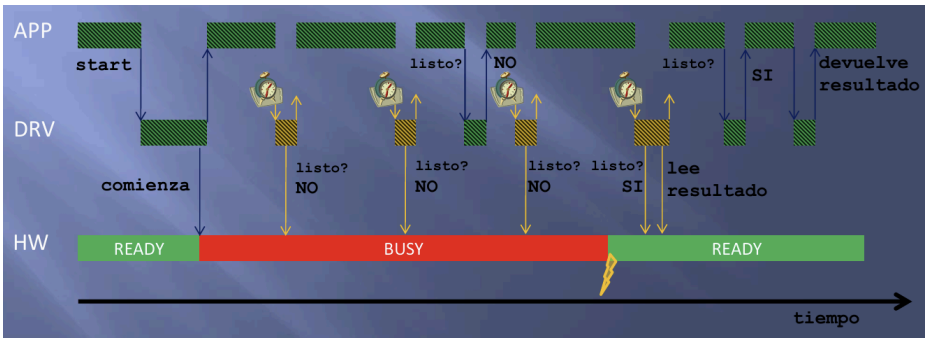
2. Polling (Consulta Continua) La Aplicación realiza una secuencia de 3 llamadas no bloqueantes: **comenzar**, **¿está listo?**, y **leer resultado**.

- **Rendimiento:** La APP puede intercalar otras tareas, pero pierde tiempo valioso volviendo a preguntar constantemente `¿listo?` al Driver.
- **Inconveniente:** El Driver actúa de simple pasamanos y la CPU consume energía inútilmente.



3. Interrupción Periódica (PISR / Hardware Polling) Un temporizador de hardware (como el **SysTick a 1 kHz**) ejecuta automáticamente la función `DRV_PISR()` a intervalos regulares para interrogar al hardware.

- **Rendimiento: Excelente.** La APP queda suspendida un tiempo ínfimo durante la interrupción y el evento se captura en RAM inmediatamente.
- **Uso:** Opción preferida para la mayoría de las interfaces humanas (botones, encoders, displays).



4. Interrupción Dedicada (ISR) El hardware envía una señal eléctrica (IRQ) a la CPU en el instante exacto en que ocurre el evento, ejecutando `DRV_ISR()`.

- **Rendimiento: Respuesta inmediata en tiempo real (mínima latencia).**
- **Uso:** Reservado para eventos ultra-rápidos que se perderían entre muestras periódicas.

Modelo	Latencia	Consumo de CPU	Caso de Uso Recomendado
Bloqueante	Nula (espera dedicada)	100% de la CPU desperdiciada	Inicializaciones sencillas de hardware al arrancar.
Polling	Variable según el loop	Alto (consultas repetitivas)	Pruebas de laboratorio o microcontroladores sin timers.

Modelo	Latencia	Consumo de CPU	Caso de Uso Recomendado
PISR (Tick)	Acotada al período (ej. 1 ms)	Mínimo (ejecución rápida en ISR)	Pulsadores, encoders, displays multiplexados, teclados.
ISR Dedicada	Inmediata (≈ nanosegundos)	Solo durante la ráfaga del evento	Lectores de banda magnética, recepción UART, encoders de motor.

Integración Segura con Callbacks y Reglas de Diseño

⚠️ REGLA ABSOLUTA DE DISEÑO: Jamás Lanzar Callbacks desde ISR / PISR

“Never ever launch a callback from inside an ISR or PISR.”
Si un driver ejecuta un **callback** inyectado por el usuario directamente dentro del contexto de una interrupción y dicho código contiene un error (bucle infinito, retardo o sección bloqueante), **todo el microcontrolador se congelará irreversiblemente**.



Arquitectura Correcta de Despacho de Eventos

Para comunicar eventos desde el hardware hacia la aplicación sin correr riesgos:

1. **La ISR / PISR solo captura los datos crudos y enciende un** Flag **en RAM.**
2. **El bucle principal (`App_Run`) consulta el estado del driver mediante funciones de servicio (`DRV_Srv`).**
3. **Al detectar el flag activo, la APP invoca la función de usuario o callback de manera segura fuera de la interrupción.**

🛠️ Código Didáctico: Arquitectura Completa de Capas y Timer PISR en C

```c
// ========================================================================
// CAPA MCAL / HAL: driver_timer.c (Gestión del SysTick y PISR)
// ========================================================================
#include "MK64F12.h"
#include <stdbool.h>

static volatile bool flag_evento_10ms = false;
static uint32_t contador_milis = 0;

// PISR: Ejecutada periódicamente a 1 kHz por la ISR del SysTick
void DRV_PISR_TimerTick(void) {
    contador_milis++;
```

```
    if ((contador_milis % 10) == 0) {
        flag_evento_10ms = true; // SOLO SE PONE EL FLAG, NO SE LLAMA AL CALLBACK
    }
}

// Servicio público para consultar estado desde la APP (Servicio No Bloqueante)
bool DRV_Srv_CheckEvent10ms(void) {
    if (flag_evento_10ms) {
        flag_evento_10ms = false; // Write-1-to-Clear lógico
        return true;
    }
    return false;
}

// ========================================================================
// CAPA APLICACIÓN: App.c (Súper-Loop y FSM)
// ========================================================================
void App_Run(void) {
    // Consulta no bloqueante de eventos del driver
    if (DRV_Srv_CheckEvent10ms()) {
        // Ejecución segura de la lógica de usuario / Callback
        FSM_Update(EV_TICK_10MS);
    }

    // El Súper-Loop continúa ejecutando otras tareas sin bloquearse
}
```

💡 Resumen de la Estrategia del Proyecto

- **SysTick (1 kHz):** Se utiliza como la base de tiempo central (Scheduler) que dispara la `PISR` para refrescar el display y procesar el antirrebote del encoder.
- **Lector Magnético (ISR Dedicada):** Captura cada bit en tiempo real por flanco en la ISR de `PORTB`. La decodificación del paquete (operación costosa) se delega a `App_Run()` fuera de la interrupción.

Clase 5

Fundamentos de Comunicación Serial vs. Paralela

La transmisión de datos en sistemas embebidos entre el microcontrolador y periféricos externos (sensores, módulos de comunicación, PCs o pantallas) se clasifica en dos arquitecturas fundamentales:

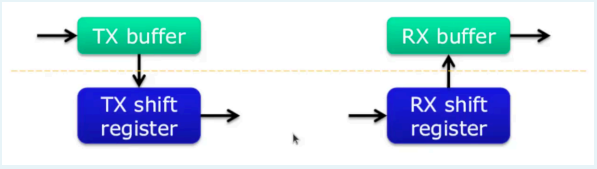
Parámetro	Comunicación Paralela	Comunicación Serial
Líneas Físicas	Múltiples líneas de datos (8, 16, 32 bits simultáneos).	Líneas reducidas (1 línea TX, 1 línea RX + GND).
Costo y Pines	Alto número de pines de I/O y cables voluminosos.	Mínimo uso de pines; conectores y cables económicos.
Distancia y Ruido	Limitado a distancias muy cortas por **skew** de reloj y acoplamiento capacitivo.	Alta inmunidad al ruido; ideal para distancias cortas y largas.
Ancho de Banda	Elevado en distancias muy cortas.	Muy alto a altas frecuencias con transceptores adecuados.

Comunicación Sincrónica vs. Asincrónica

- **Sincrónica (SPI, I2C):** Existe una línea física explícita de reloj (**Clock**) provista por un máster para sincronizar la lectura de los bits en emisor y receptor.
- **Asincrónica (UART):** No existe línea física de reloj. Emisor y receptor acuerdan previamente una velocidad fija (**Baud Rate**) y sincronizan sus relojes locales mediante el formato de una trama estructurada (**Start/Stop bits**).

> 💡 **Bit Banging vs. Periférico Hardware**
>
> - **Bit Banging:** Simular la transmisión serie por software conmutando manualmente pines GPIO mediante delays o timers. Consume valiosos ciclos de CPU y es propenso a fluctuaciones de tiempo (**jitter**).
> - **Periférico Dedicado (UART):** Módulo de hardware integrado que realiza la conversión paralelo-serie y serie-paralelo de forma autónoma mediante registros de desplazamiento y buffers, liberando a la CPU.

Buffers

> 💡 **Hardware Buffers**
>
> 
>
>
> Evita que un dispositivo rápido (como la CPU) se detenga o pierda información al comunicarse con otro más lento

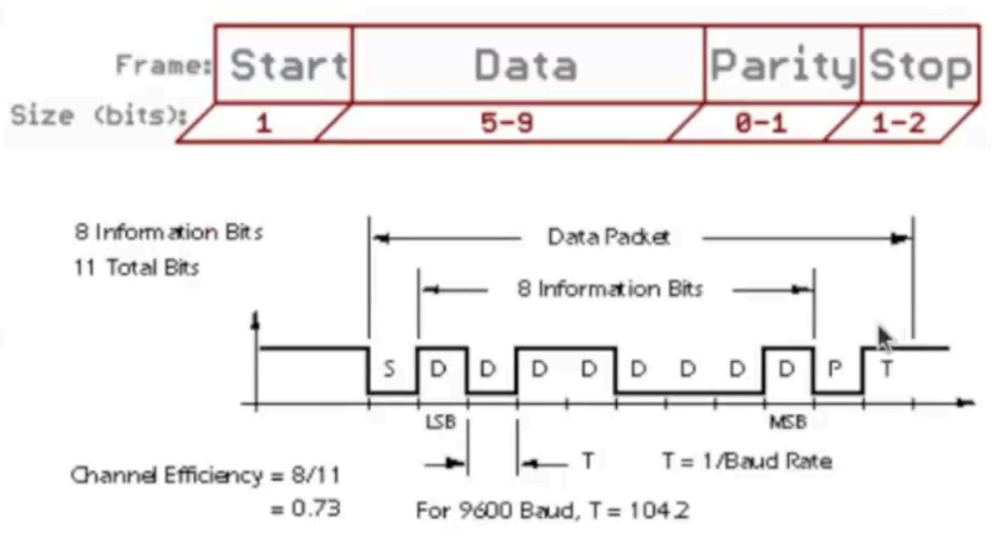
> 💡 **Software Buffers**
>
> Emular en el driver del protocolo serie un TX/RX buffer de N bytes.

Anatomía Completa de la Trama UART (Protocolo 8N1)

En reposo, la línea de transmisión se mantiene en nivel lógico ALTO (1). La trama estándar **8N1** (8 bits de datos, sin paridad, 1 bit de stop) se compone secuencialmente de:

⚙️ Campos Secuenciales de la Trama UART

1. **Línea en Reposo (Idle):** Permanece en nivel lógico **ALTO (1 / V_{DD})**.
2. **Bit de Start:** Un pulso de nivel **BAJO (0 / GND)** de duración exacta de 1 tiempo de bit (T_{bit}) que avisa al receptor del inicio de datos.
3. **Carga Útil (Payload / Data Bits):** Se transmiten de 5 a 8 bits de datos, enviando siempre el **bit menos significativo (LSB) primero**.
4. **Bit de Paridad (Opcional - Par/Impar/Ninguna):** Bit de control para detección de errores simples de bit.
5. **Bit(s) de Stop:** Uno o dos bits en nivel **ALTO (1 / V_{DD})** que marcan el cierre de la trama y retornan la línea a reposo.



Eficiencia de Canal y Muestreo por Sobremuestreo

En una trama estándar 8N1 se envían 10 bits totales para transmitir 8 bits útiles de información:

$$\text{Eficiencia} = \frac{8 \text{ bits útiles}}{10 \text{ bits totales}} = 80\ \%$$

$$T_{bit} = \frac{1}{\text{Baud Rate}} \implies \text{A 9600 bps: } T_{bit} = \frac{1}{9600} \approx 104.16\mu \text{ s}$$

⚠️ Sobremuestreo 16x y Tolerancia de Sincronismo

El receptor detecta el flanco descendente del bit de Start y sobremuestrea la línea $16\times$ más rápido que el Baud Rate. Muestrea el nivel de tensión en el **centro teórico de cada bit ($0.5 \times T_{bit}$)** para maximizar el margen de ruido.

Tolerancia Máxima: Dado que emisor y receptor poseen cristales independientes, un desfasaje acumulado mayor al $2.5\ \%$ entre ambos relojes provoca la lectura incorrecta del bit de Stop, generando un error de trama (**Framing Error - FE**).

Bitrate vs Baurate

- **BitRate**= bits/Seg (bps)
- **BaudRate**= Numero de cambios en la señal/seg o símbolos /seg
- **Símbolo** = Uno de los diferentes valores en la tensión,frecuencia o Fase

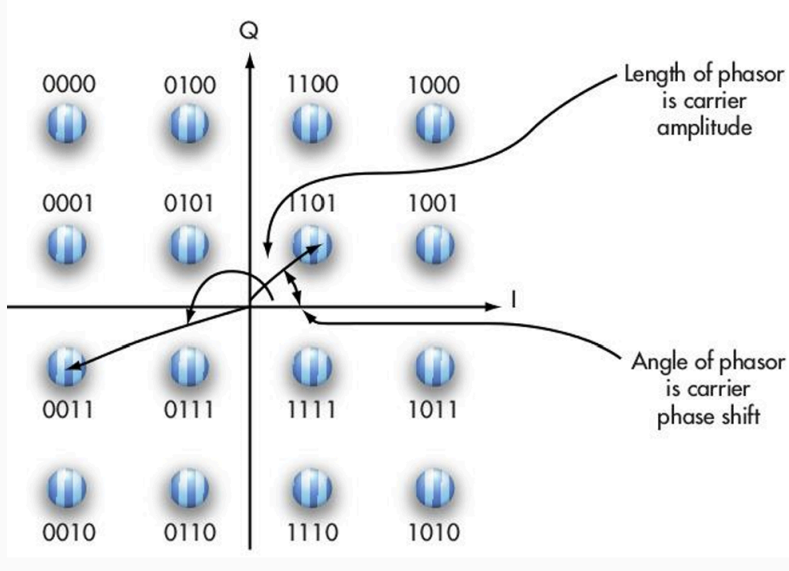


Figura 18: Ejemplo de baudrate con amplitud y fase. 4 bit / Símbolo => Bitrate=4 x baudrate

Capa Física, Estándares Eléctricos y Control de Flujo

La salida nativa del microcontrolador opera en niveles **TTL/CMOS (0V a 3.3V)**. Para comunicarse con computadoras o equipos industriales se requieren adaptadores de capa física:

Estándar	Niveles de Tensión	Características / Uso
TTL / CMOS	$0\ \text{V} = 0$ $3.3\ \text{V} / 5\ \text{V} = 1$	Conexiones punto a punto dentro de la misma placa a corta distancia.
RS-232	$+3\ \text{V}\ \text{ a }\ +15\ \text{V} = 0$ (Space), $-3\ \text{V}\ \text{ a }\ -15\ \text{V} = 1$ (Mark)	Lógica invertida de alta tensión. Requiere transceptor como MAX232. Inmunidad a interferencias.
USB-Serial (OpenSDA)	Conversión USB CDC Virtual COM	Integrado en FRDM-K64F. Permite ver la UART en la PC mediante terminales (PuTTY, TeraTerm).
Comandos AT	Lógica de texto sobre UART (`AT`, `AT+CSQ`)	Estándar de configuración para módulos Bluetooth (HC-05), GSM (SIM800) y GPS.

Control de Flujo por Software (XON / XOFF)

Cuando el receptor no puede procesar los datos a la velocidad que los recibe, utiliza **Control de Flujo por Software**:

- **XOFF (`0x13` / Ctrl+S):** Enviado por el receptor para ordenar al emisor pausar la transmisión.
- **XON (`0x11` / Ctrl+Q):** Enviado por el receptor cuando vuelve a tener espacio disponible en el buffer para reanudar la transmisión.

Se deja un margen al contemplar el delay del flag de `Xoff`.

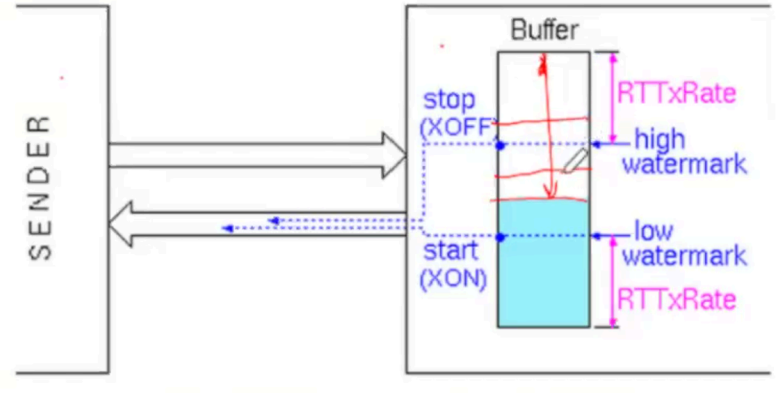


Figura 19: Protocolo Xon/Xoff

Conversores

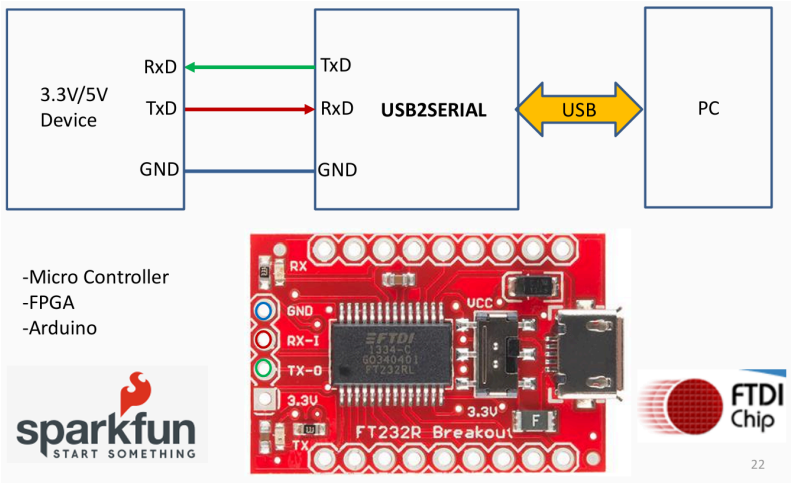


Figura 20: Adaptador

Una manera de hacer un conversor casero, seria shortear un esp32 entre `GND` y `RESET` para generar una alta impendanica en el `CPU` para que la señal que entre por el USB va ya directo a `TX` y `RX`.

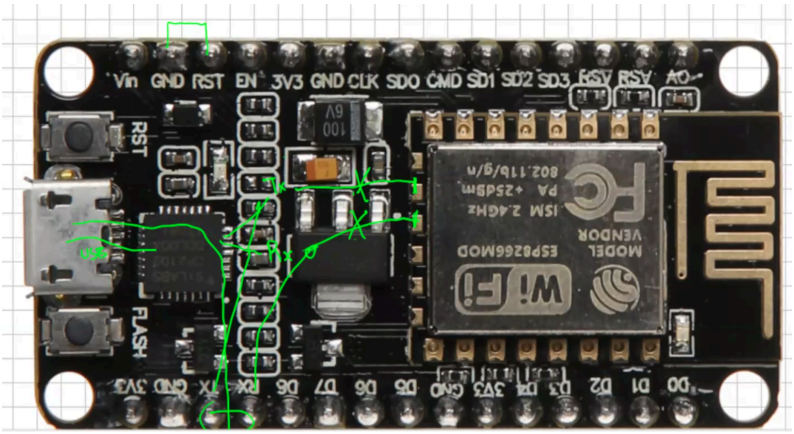


Figura 21: Alternativa barata

Colas Circulares (Ring Buffers) e Interrupciones

Existe una clara **asimetría temporal** entre la transmisión y la recepción:

- **Recepción (RX):** Es crítica e impredecible. Si no se lee el dato de inmediato cuando llega, el siguiente byte entrante lo sobrescribirá, causando un error de sobreescritura (**Overrun Error - OR**).
- **Transmisión (TX):** No es crítica. Si se demora el envío, únicamente se pierde velocidad de transmisión.

> 💡 **Solución: Colas Circulares Orientadas a Interrupción**
>
> Para evitar código bloqueante, la ISR de recepción guarda los bytes inmediatamente en una **Cola Circular (Ring Buffer)** en memoria RAM. El programa principal (**Main Loop**) consume los datos de la cola a su propio ritmo.

Código C Didáctico de una Cola Circular (`RingBuffer`)

```c
#define RING_BUFFER_SIZE 64

typedef struct {
    uint8_t buffer[RING_BUFFER_SIZE];
    uint16_t in;    // Puntero de entrada (escritura ISR)
    uint16_t out;   // Puntero de salida (lectura APP)
    uint16_t count; // Cantidad de elementos almacenados
} RingBuffer_t;

void RingBuffer_Init(RingBuffer_t *rb) {
    rb->in = 0;
    rb->out = 0;
    rb->count = 0;
}

bool RingBuffer_Push(RingBuffer_t *rb, uint8_t data) {
    if (rb->count >= RING_BUFFER_SIZE) return false; // Buffer Lleno
    rb->buffer[rb->in] = data;
    rb->in = (rb->in + 1) % RING_BUFFER_SIZE;
    rb->count++;
    return true;
}

bool RingBuffer_Pull(RingBuffer_t *rb, uint8_t *data) {
    if (rb->count == 0) return false; // Buffer Vacío
    *data = rb->buffer[rb->out];
    rb->out = (rb->out + 1) % RING_BUFFER_SIZE;
    rb->count--;
    return true;
}
```

Mapeo de Registros y Cálculo Fraccional de Baud Rate en Kinetis K64F

El Kinetis K64F cuenta con 6 módulos UART (`UART0` a `UART5`):
- `UART0` y `UART1`: Alimentados desde el **System Clock** (f_{core} hasta 100 MHz).
- `UART2` a `UART5`: Alimentados desde el **Bus Clock** (f_{bus} hasta 50 MHz).

> ⚙️ **Fórmula Exacta del Baud Rate Fraccional**
>
> El generador de Baud Rate utiliza un divisor grueso de 13 bits (`SBR` dividido entre los registros `BDH` y `BDL`) y un ajuste fino fraccional de 5 bits (`BRFA` en el registro `C4`):
>
> $$\text{Baud Rate} = \frac{\text{UART_CLK}}{16 \times \left(\text{SBR} + \frac{\text{BRFA}}{32}\right)}$$
>
> - **Cálculo de SBR:** $\text{SBR} = \left\lfloor \frac{\text{UART_CLK}}{16 \times \text{ Baud Rate}} \right\rfloor$
> - **Cálculo de BRFA:** $\text{BRFA} = \left\lfloor \left(\left(\frac{\text{UART_CLK}}{16 \times \text{ Baud Rate}} \right) - \text{SBR} \right) \times 32 \right\rceil$

```c
void UART_Init (void){
// Note: 5.6 Clock Gating page 192
// Any bus access to a peripheral that has its clock disabled generates an error termination.
    SIM->SCGC5 |= SIM_SCGC5_PORTB_MASK;

    SIM->SCGC4 |= SIM_SCGC4_UART0_MASK;
    SIM->SCGC4 |= SIM_SCGC4_UART1_MASK;
    SIM->SCGC4 |= SIM_SCGC4_UART2_MASK;
    SIM->SCGC4 |= SIM_SCGC4_UART3_MASK;
    SIM->SCGC1 |= SIM_SCGC1_UART4_MASK;
    SIM->SCGC1 |= SIM_SCGC1_UART5_MASK;

    NVIC_EnableIRQ(UART0_RX_TX_IRQn);
    NVIC_EnableIRQ(UART1_RX_TX_IRQn);
    NVIC_EnableIRQ(UART2_RX_TX_IRQn);
    NVIC_EnableIRQ(UART3_RX_TX_IRQn);
    NVIC_EnableIRQ(UART4_RX_TX_IRQn);
    NVIC_EnableIRQ(UART5_RX_TX_IRQn);

    //UART0 Set UART Speed
    UART_SetBaudRate(UART0, UART_HAL_DEFAULT_BAUDRATE);

    //Configure UART0 TX and RX PINS
    PORTB->PCR[UART0_TX_PIN]=0x0; //Clear all bits
    PORTB->PCR[UART0_TX_PIN]|=PORT_PCR_MUX(PORT_mAlt3);    //Set MUX to UART0
    PORTB->PCR[UART0_TX_PIN]|=PORT_PCR_IRQC(PORT_eDisabled); //Disable [interrupts]

  //UART0 Baudrate Setup
    UART_SetBaudRate (UART0, 9600);

  //Enable UART0 Xmiter and Rcvr
  UART0->C2=UART_C2_TE_MASK | UART_C2_RE_MASK;
}

void UART_SetBaudRate (UART_Type *uart, uint32_t baudrate){
  uint16_t sbr, brfa;
  uint32_t clock;

  clock = ((uart == UART0) || (uart == UART1))?(__CORE_CLOCK__):(__CORE_CLOCK__ >> 1);

  baudrate = ((baudrate == 0)?(UART_HAL_DEFAULT_BAUDRATE):
      ((baudrate > 0x1FFF)?(UART_HAL_DEFAULT_BAUDRATE):(baudrate)));

  sbr = clock / (baudrate << 4);               // sbr = clock/(Baudrate x 16)
  brfa = (clock << 1) / baudrate - (sbr << 5); // brfa = 2*Clock/baudrate - 32*sbr

  uart->BDH = UART_BDH_SBR(sbr >> 8);
  uart->BDL = UART_BDL_SBR(sbr);
  uart->C4 = (uart->C4 & ~UART_C4_BRFA_MASK) | UART_C4_BRFA(brfa);
}
```

Control de Registros, FIFOs y Driver UART Completo en C

Registros de Control y Estado

- **`UARTx_C1`:** Configura formato de trama (8/9 bits, paridad).
- **`UARTx_C2`:** Habilita transmisor (`TE`), receptor (`RE`), interrupción por transmisión libre (`TIE`) e interrupción por recepción llena (`RIE`).
- **`UARTx_S1`:** Almacena los flags de estado principales (`TDRE`, `TC`, `RDRF`, `IDLE`, `OR`, `NF`, `FE`, `PF`).
- **`UARTx_D`:** Registro de datos de entrada/salida.

> ⚠️ **Secuencia de Doble Lectura para Limpiar Banderas**
>
> En los módulos UART de Kinetis, la bandera `RDRF` (o `TDRE`) **no se limpia escribiendo un 1**. Se limpia ejecutando la secuencia estricta por hardware:
> 1. **Primero:** Leer el registro de estado `UARTx_S1`.
> 2. **Segundo:** Leer (o escribir) el registro de datos `UARTx_D`.

Configuración de FIFOs de Hardware Las UARTs disponen de FIFOs de hardware (`PFIFO`, `TWFIFO`, `RWFIFO`) que permiten configurar marcas de agua (**watermarks**). Por ejemplo, `RWFIFO = 1` genera la interrupción `RDRF` en cuanto ingresa al menos 1 byte en la FIFO de recepción.

Código C Completo del Driver UART0 Orientado a Interrupciones

```c
#include "MK64F12.h"
#include <stdbool.h>

static RingBuffer_t rx_buffer;

void UART0_Init(uint32_t baud_rate) {
    // 1. Clock Gating para UART0 y PORTB
    SIM->SCGC4 |= SIM_SCGC4_UART0_MASK;
    SIM->SCGC5 |= SIM_SCGC5_PORTB_MASK;

    // 2. Multiplexación de pines PTB16 (RX) y PTB17 (TX) en ALT3
    PORTB->PCR[16] = PORT_PCR_MUX(3);
    PORTB->PCR[17] = PORT_PCR_MUX(3);

    // 3. Deshabilitar Transmisor y Receptor antes de configurar
    UART0->C2 &= ~(UART_C2_TE_MASK | UART_C2_RE_MASK);

    // 4. Configurar Baud Rate (Ejemplo simplificado para 9600 bps a 100 MHz)
    uint16_t sbr = (uint16_t)(100000000 / (16 * baud_rate));
    UART0->BDH = (sbr >> 8) & UART_BDH_SBR_MASK;
    UART0->BDL = sbr & UART_BDL_SBR_MASK;
    UART0->C4 = (UART0->C4 & ~UART_C4_BRFA_MASK) | UART_C4_BRFA(16); // BRFA ajustado

    // 5. Inicializar Buffer Circular y Habilitar Interrupciones en C2 y NVIC
    RingBuffer_Init(&rx_buffer);
    UART0->C2 |= UART_C2_RIE_MASK;                  // Habilitar IRQ por Recepción
    UART0->C2 |= UART_C2_TE_MASK | UART_C2_RE_MASK; // Habilitar TX y RX

    NVIC_SetPriority(UART0_RX_TX_IRQn, 2);
    NVIC_EnableIRQ(UART0_RX_TX_IRQn);
}

// Rutina de Servicio de Interrupción (ISR Compartida)
void UART0_RX_TX_IRQHandler(void) {
    uint8_t status = UART0->S1;

    // Si hay un dato recibido en el registro D
    if (status & UART_S1_RDRF_MASK) {
        uint8_t data = UART0->D; // La lectura de S1 + D limpia RDRF automáticamente
        RingBuffer_Push(&rx_buffer, data);
    }
}

// Servicio de lectura no bloqueante para la Aplicación
bool UART0_ReadByte(uint8_t *data) {
    return RingBuffer_Pull(&rx_buffer, data);
}
```

Clase 6

Protocolo SPI (Serial Peripheral Interface)

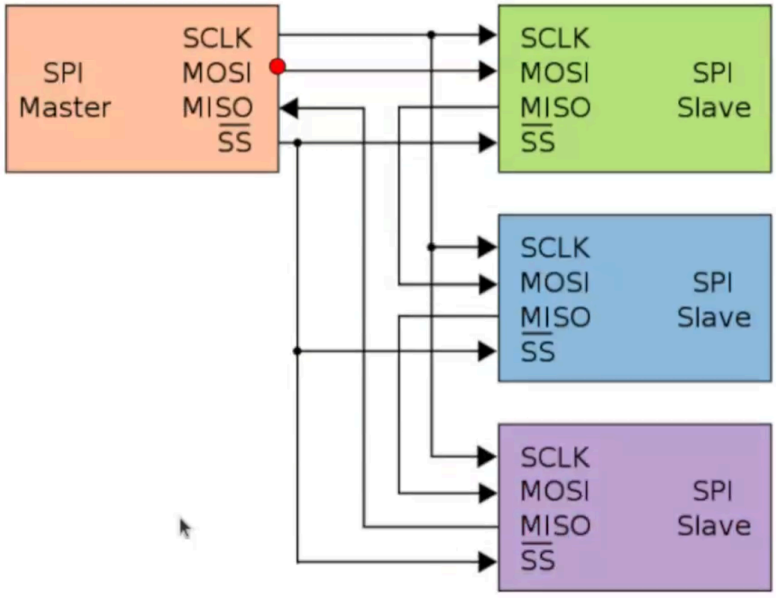
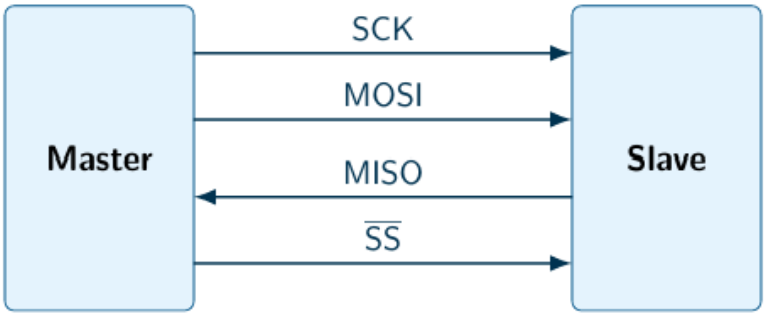


Figura 22: Protocolo SPI

Arquitectura y Señales Físicas

SPI es un protocolo serie síncrono diseñado para comunicaciones de corta distancia entre periféricos y microcontroladores. Opera bajo una arquitectura estricta de maestro-esclavo, donde el dispositivo maestro siempre inicia la transferencia y genera la señal de reloj para sincronizar los datos.

Físicamente, un enlace SPI utiliza cuatro señales:
- **SCK (Serial Clock):** Reloj generado exclusivamente por el maestro.
- **MOSI (Master Out Slave In):** Línea de datos desde el maestro hacia el esclavo.
- **MISO (Master In Slave Out):** Línea de datos desde el esclavo hacia el maestro.
- **SS (Slave Select):** Señal de habilitación, típicamente activa en bajo, con la que el maestro selecciona específicamente el esclavo activo.



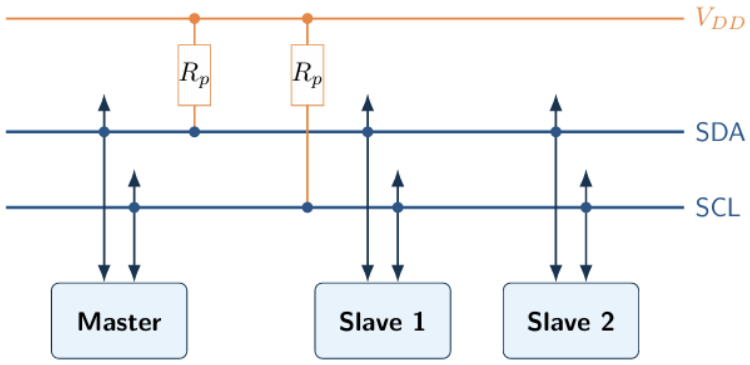
> ⚙️ **Mecanismo de Shift Registers**
>
> Internamente, la comunicación SPI funciona como un gran anillo de registros de desplazamiento (**shift registers**). Tanto el maestro como el esclavo poseen un registro de desplazamiento; durante cada ciclo de reloj, el maestro empuja un bit por MOSI hacia el esclavo, y simultáneamente el esclavo empuja un bit por MISO hacia el maestro. Toda transmisión de datos implica inherentemente una recepción paralela.

Topologías de Red

> 💡 **Topología en Bus:**
>
> Todos los esclavos comparten en paralelo las líneas SCK, MISO y MOSI. El maestro utiliza múltiples pines de SS (uno dedicado de forma individual por cada esclavo). Las salidas MISO de los esclavos deshabilitados deben permanecer obligatoriamente en estado de alta impedancia ($\mathrm{Hi} - Z$).

> 💡 **Topología en Cascada (Daisy Chain):**
>
> Se utiliza un único pin SS compartido para todos los esclavos. La línea MOSI del maestro entra al primer esclavo, la salida MISO del primero se conecta a la entrada MOSI del segundo, y así sucesivamente, hasta que el MISO del último esclavo retorna al maestro.

Protocolo I2C (Inter-Integrated Circuit)



Arquitectura Eléctrica y Wired-AND

I2C es un bus serie síncrono, multimaestro y multiesclavo de baja velocidad (típicamente entre 100 kbps y 400 kbps) que utiliza únicamente dos hilos bidireccionales y una referencia de masa compartida (GND):

> ⚙️ **SDA (Serial Data)**
>
> Línea de datos bidireccional.

> ⚙️ **SCL (Serial Clock)**
>
> Línea de reloj bidireccional controlada por el máster activo.

Todos los pines de los dispositivos están configurados en drenador o colector abierto (**Open-Drain**) y comparten resistencias de **Pull-Up** externas conectadas a V_{DD} (típicamente $1.8\ \mathrm{k\Omega}$ a $10\ \mathrm{k\Omega}$). Esta configuración forma una compuerta **Wired-AND** de hardware: la línea solo sube a ALTO si todos los dispositivos dejan su salida en alta impedancia ($\mathrm{Hi} - Z$), lo que previene cortocircuitos si dos equipos transmiten simultáneamente.

En el Kinetis K64F, es necesario activar el bit `ODE` escribiendo un “1” en el registro `PORTx_PCRn` correspondiente.

Formas de Onda y Reglas del Protocolo I2C

Condición / Fase	Comportamiento de Señal (SDA / SCL)	Función en la Trama
START (S)	Flanco descendente de SDA mientras SCL está en ALTO.	Inicia la transferencia y toma el control del bus.
STOP (P)	Flanco ascendente de SDA mientras SCL está en ALTO.	Finaliza la transferencia y libera el bus (Idle).
REPEATED START (Sr)	Condición START generada sin emitir un STOP previo.	Permite cambiar de dirección TX/RX sin perder el control del bus ni reiniciar la máquina de estados del esclavo.

Condición / Fase	Comportamiento de Señal (SDA / SCL)	Función en la Trama
Transferencia Datos	SDA solo puede cambiar cuando SCL está en BAJO. Se lee en el nivel ALTO de SCL.	Transmite 8 bits (**MSB primero**).
ACK / NACK	El receptor fuerza SDA a BAJO (ACK = 0) o lo deja libre (NACK = 1) en el 9º pulso de SCL.	Confirmación de recepción del byte por parte del receptor.

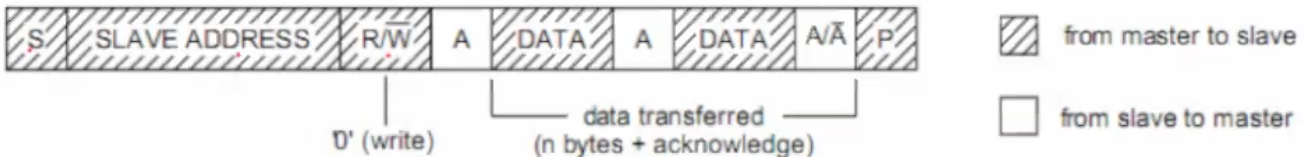


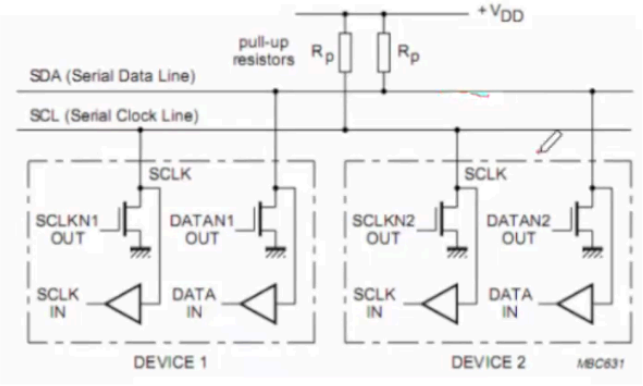
Figura 25: Protocolo de Escritura

Mecanismos Avanzados

⚙️ Clock Stretching y Arbitraje

Clock Stretching: Si un esclavo necesita tiempo adicional para procesar un byte recibido, puede mantener forzosamente SCL en nivel bajo. El maestro detecta este estado de espera (**wait state**) y pausa la transmisión hasta que el esclavo libera SCL.

Arbitraje de Bus: Si dos maestros emiten un START simultáneamente, ambos monitorean la línea SDA mientras transmiten. Si un maestro impone un '1' (recesivo) pero lee un '0' (dominante forzado por otro nodo), detecta una colisión, pierde el arbitraje y aborta su transmisión inmediatamente sin corromper el mensaje.



My Device	Other Device	Bus
0V (**0**)	don't care (**X**)	0V (**0**)
Hi-Z (**1**)	0V (**0**)	0V (**0**)
Hi-Z (**1**)	Hi-Z (**1**)	$+V_{DD}$ (**1**)

Figura 26: lógica Wired-AND (1 Blando y 0 Dominante).

Driver I2C en Kinetis K64F

El microcontrolador K64F cuenta con los módulos `I2C0`, `I2C1` e `I2C2`. Registros principales:

- `I2Cx->A1`: Dirección propia de esclavo (7 bits).
- `I2Cx->F`: Divisor de frecuencia para SCL a partir del Bus Clock.
- `I2Cx->C1`: Control principal (`IICEN`, `IICIE`, `MST`, `TX`, `TXAK`, `RSTA`).
- `I2Cx->S`: Estado (`TCF`, `BUSY`, `ARBL`, `IICIF` **w1c**, `RXAK`).
- `I2Cx->D`: Registro de datos de entrada/salida.

Código C: Driver I2C Máster (Inicialización, Escritura y Lectura)

```c
#include "MK64F12.h"
#include <stdbool.h>

// 1. Inicialización del Módulo I2C0
void I2C0_Init(void) {
```

```
    // A) Clock Gating para I2C0 y PORTE
    SIM->SCGC4 |= SIM_SCGC4_I2C0_MASK;
    SIM->SCGC5 |= SIM_SCGC5_PORTE_MASK;

    // B) Configurar MUX = ALT5 (I2C) y activar Open-Drain (ODE=1) en PTE24 (SCL) y PTE25 (SDA)
    PORTE->PCR[24] = PORT_PCR_MUX(5) | PORT_PCR_ODE_MASK;
    PORTE->PCR[25] = PORT_PCR_MUX(5) | PORT_PCR_ODE_MASK;

    // C) Configurar Frecuencia de SCL ~ 100 kHz (con Bus Clock de 50 MHz)
    I2C0->F = 0x14;

    // D) Habilitar el Módulo I2C
    I2C0->C1 = I2C_C1_IICEN_MASK;
}

// 2. Transmisión Máster (Escritura de un Byte a un Esclavo)
bool I2C0_WriteByte(uint8_t slave_addr, uint8_t reg_addr, uint8_t data) {
    while (I2C0->S & I2C_S_BUSY_MASK);

    I2C0->C1 |= I2C_C1_MST_MASK | I2C_C1_TX_MASK;
    I2C0->D = (slave_addr << 1) | 0;
    while (!(I2C0->S & I2C_S_IICIF_MASK));
    I2C0->S |= I2C_S_IICIF_MASK;

    if (I2C0->S & I2C_S_RXAK_MASK) {
        I2C0->C1 &= ~I2C_C1_MST_MASK; // Generar STOP
        return false;
    }

    I2C0->D = reg_addr;
    while (!(I2C0->S & I2C_S_IICIF_MASK));
    I2C0->S |= I2C_S_IICIF_MASK;

    I2C0->D = data;
    while (!(I2C0->S & I2C_S_IICIF_MASK));
    I2C0->S |= I2C_S_IICIF_MASK;

    I2C0->C1 &= ~(I2C_C1_MST_MASK | I2C_C1_TX_MASK);
    return true;
}

// 3. Recepción Máster (Lectura de un Byte desde un Esclavo)
uint8_t I2C0_ReadByte(uint8_t slave_addr, uint8_t reg_addr) {
    uint8_t dummy, data;

    I2C0->C1 |= I2C_C1_MST_MASK | I2C_C1_TX_MASK;
    I2C0->D = (slave_addr << 1) | 0;
    while (!(I2C0->S & I2C_S_IICIF_MASK));
    I2C0->S |= I2C_S_IICIF_MASK;

    I2C0->D = reg_addr;
    while (!(I2C0->S & I2C_S_IICIF_MASK));
    I2C0->S |= I2C_S_IICIF_MASK;

    // Repeated Start
    I2C0->C1 |= I2C_C1_RSTA_MASK;
    I2C0->D = (slave_addr << 1) | 1; // R/W = 1
    while (!(I2C0->S & I2C_S_IICIF_MASK));
    I2C0->S |= I2C_S_IICIF_MASK;

    I2C0->C1 &= ~I2C_C1_TX_MASK;
    I2C0->C1 |= I2C_C1_TXAK_MASK; // Responder con NACK
```

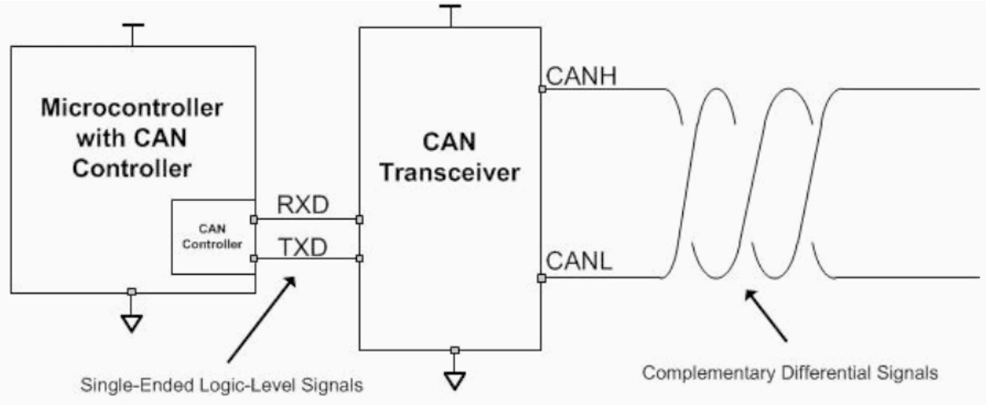
```
    dummy = I2C0->D; // Lectura dummy
    while (!(I2C0->S & I2C_S_IICIF_MASK));
    I2C0->S |= I2C_S_IICIF_MASK;

    I2C0->C1 &= ~(I2C_C1_MST_MASK | I2C_C1_TXAK_MASK);
    data = I2C0->D; // Lectura real

    return data;
}
```

Clase 7

Protocolo CAN (Controller Area Network)



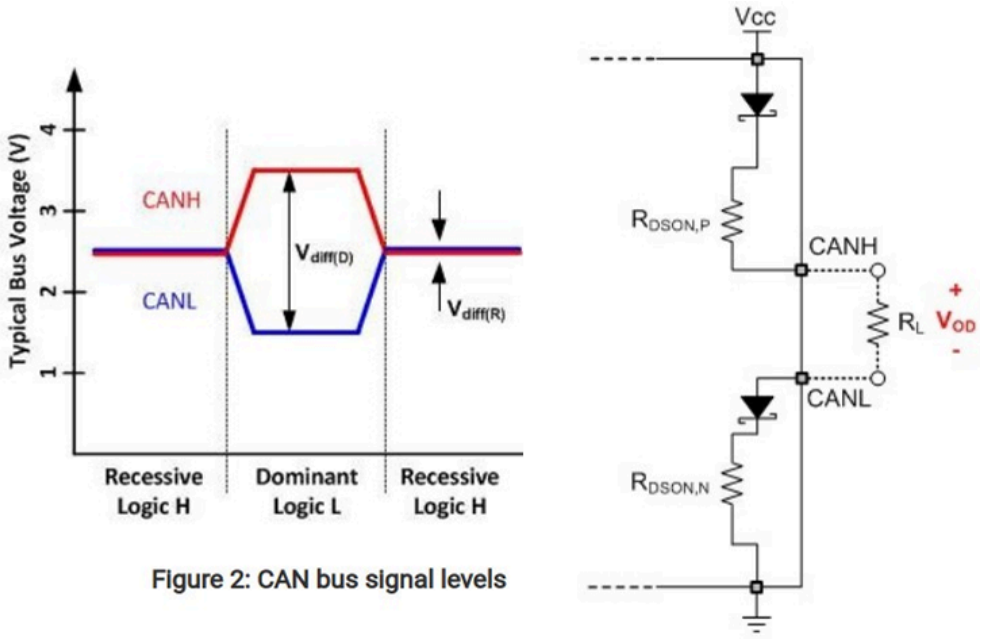


Figure 2: CAN bus signal levels

Arquitectura y Características Principales

CAN es un estándar de bus serie asíncrono, multimaestro y altamente robusto diseñado originalmente por Bosch para aplicaciones automotrices e industriales (ISO 11898). Permite que múltiples Unidades de Control Electrónico (ECUs) se comuniquen sobre un bus común sin necesidad de un nodo maestro central o controlador host.

Características fundamentales del protocolo:

- **Multimaestro / Asíncrono:** Cualquier nodo puede iniciar la transmisión cuando el bus está libre (**Idle**).
- **Orientado a Mensajes:** Los mensajes no contienen direcciones de nodo origen o destino; en su lugar, incorporan un **Identificador (ID)** que define la naturaleza del dato y su prioridad.
- **Inmunidad al Ruido:** Utiliza señalización diferencial sobre par trenzado.
- **Arbitraje No Destructivo:** Resuelve colisiones en tiempo real mediante lógica de bus dominante/recesivo (**Wired-AND**).
- **Detección y Manejo de Errores:** Lógica integrada por hardware para contención y aislamiento de fallas.

Capa Física y Señal Diferencial

A nivel eléctrico, el bus CAN se compone de un par de cables trenzados (**twisted pair**) denominados **CANH** ($\mathrm{CAN}+$ y **CANL** ($\mathrm{CAN}-$, acompañados por una referencia común de masa (GND). Para evitar reflexiones de

onda electromagnética en altas frecuencias, el bus exige **resistencias de terminación de** 120Ω instaladas en ambos extremos físicos de la línea.

El bus opera bajo dos estados eléctricos distintos que implementan la lógica **Wired-AND**:

> ⚙️ **Estado Recesivo ('1' Lógico)**
>
> - Transceptores en alta impedancia (apagados).
> - Las líneas descansan pasivamente a través de resistencias **pull-middle**.
> - Voltaje: $V_{\text{CANH}} = 2.5\ \text{V}$, $V_{\text{CANL}} = 2.5\ \text{V}$.
> - Tensión diferencial: $V_{\text{diff}} = V_{\text{CANH}} - V_{\text{CANL}} = 0\ \text{V}$.

> ⚙️ **Estado Dominante ('0' Lógico)**
>
> - Transceptores activos excitando la línea.
> - Se fuerzan niveles de tensión separados.
> - Voltaje: $V_{\text{CANH}} = 3.5\ \text{V}$, $V_{\text{CANL}} = 1.5\ \text{V}$.
> - Tensión diferencial: $V_{\text{diff}} = 2.0\ \text{V}$.

> 💡 **Lógica Wired-AND y Colisión**
>
> Al ser un bus cableado en configuración **Wired-AND**, si dos nodos intentan transmitir simultáneamente y uno de ellos impone un estado **Dominante ('0')** mientras el otro envía un **Recesivo ('1')**, el estado dominante sobrescribe físicamente al recesivo en todo el bus. Esto permite resolver colisiones sin destruir el mensaje dominante.

Estructura de Trama, Arbitraje y Bit Stuffing

7.1.3.1 Trama de Datos Estándar (Standard Data Frame)

Un **Data Frame** Estándar (CAN 2.0A) se compone de los siguientes campos secuenciales:

Campo	Longitud	Descripción
SOF (Start of Frame)	1 bit	Bit dominante ($1 \to 0$) que marca el inicio de trama y sincroniza a todos los nodos.
Arbitration Field	12 bits	Contiene el **Identificador (ID)** de 11 bits y el bit **RTR** (0 = Data Frame, 1 = Remote Frame).
Control Field	6 bits	Incluye bits de extensión/reserva (`IDE`, `r0`) y el `DLC` (Data Length Code, 4 bits: $0..8$ bytes).
Data Field	$0..8$ bytes	Carga útil (**payload**) del mensaje ($0..64$ bits).
CRC Field	16 bits	Secuencia de verificación de redundancia cíclica de 15 bits + 1 bit `CRC Delimiter` (recesivo).
ACK Field	2 bits	`ACK Slot` (1 bit, enviado recesivo por el emisor y forzado a dominante por receptores con CRC válido) + `ACK Delimiter` (1 bit recesivo).
EOF (End of Frame)	7 bits	Fin de trama compuesto por 7 bits recesivos consecutivos.
ITM (Intermission)	3 bits	Espacio de descanso recesivo obligatorio entre tramas (**Interframe Space**).

7.1.3.2 Arbitraje No Destructivo (CSMA/CD + AMP)

Cuando el bus entra en estado pasivo (**Idle**), cualquier nodo con un mensaje pendiente puede emitir el bit `SOF`. Si dos o más nodos inician la transmisión en el mismo instante:

1. Cada nodo transmite los bits de su `ID` mientras monitorea continuamente el nivel físico del bus.
2. Si un nodo envía un bit **recesivo ('1')** en su ID pero lee un nivel **dominante ('0')** en el bus, detecta que otro mensaje en competencia tiene mayor prioridad.
3. El nodo perdedor retira su transmisor inmediatamente, pasa a modo de solo escucha (**listening only**) y aguarda a que el bus vuelva a quedar libre.

4. **Regla de prioridad:** Cuanto **menor sea el valor numérico del ID**, mayor es su prioridad en la red (el bit dominante '0' predomina en las primeras posiciones).

7.1.3.3 Regla de Bit Stuffing

Debido a que CAN es un protocolo asíncrono sin línea física de reloj, los receptores dependen de las transiciones de voltaje ($1 \rightarrow 0$) para sincronizar sus relojes internos.

> ⚠️ **Mecanismo de Bit Stuffing**
>
> Si el transmisor detecta **5 bits consecutivos de la misma polaridad** en el flujo de datos, el hardware inserta automáticamente **1 bit de relleno (Stuff Bit)** de polaridad opuesta antes de enviar el siguiente bit. Los nodos receptores remueven este bit de relleno de forma transparente antes de procesar la trama.
>
> Esta regla se aplica estrictamente desde el bit `SOF` hasta el final de la secuencia `CRC` (se excluyen el `CRC Delimiter`, el campo `ACK` y el `EOF`).

Base de Tiempo y Sincronización (Nominal Bit Time)

7.1.4.1 División del Bit en Time Quanta (t_q

En redes CAN, la duración de cada bit (**Nominal Bit Time**) se subdivide en N unidades discretas de tiempo denominadas **Time Quanta** (t_q. La duración de 1 t_q se deriva del reloj interno del microcontrolador (f_{osc} mediante el divisor **Baud Rate Prescaler** (BRP.

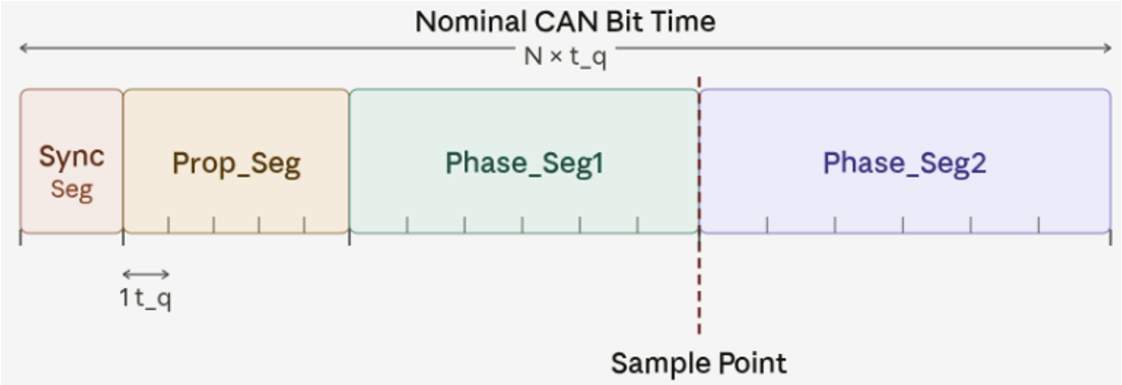


Figura 29: Estructura interna del Nominal Bit Time en sus cuatro segmentos de Time Quanta

El tiempo de bit se divide exactamente en cuatro segmentos configurables:

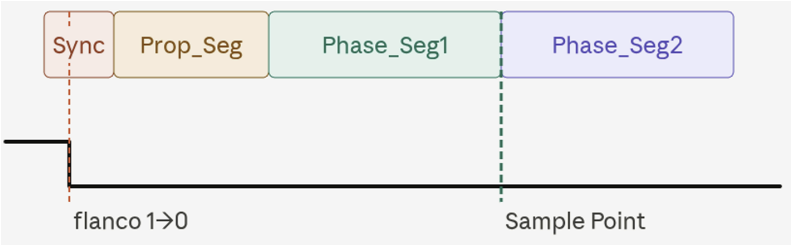
1. **Sync_Seg (Synchronization Segment):** Fijado siempre en 1 t_q. Es el intervalo donde se espera que ocurra el flanco descendente ($1 \rightarrow 0$ que marca el inicio del bit.
2. **Prop_Seg (Propagation Segment):** Configurable de 1 a 8 t_q. Compensa los retrasos físicos de propagación de las ondas electromagnéticas a lo largo del cable y los retardo internos de los transceptores.
3. **Phase_Seg1 (Phase Buffer Segment 1):** Configurable de 1 a 8 t_q. Se ubica inmediatamente antes del **Sample Point**. Se alarga en la resincronización si el flanco llega tarde.
4. **Phase_Seg2 (Phase Buffer Segment 2):** Configurable de 1 a 8 t_q. Se ubica inmediatamente después del **Sample Point**. Se acorta en la resincronización si el flanco llega temprano.

> 💡 **Sample Point (Punto de Muestreo)**
>
> El **Sample Point** es el instante exacto, situado justo al final de `Phase_Seg1`, en el que el nodo evalúa el estado lógico del bus (dominante o recesivo). Se suele configurar típicamente entre el 75% y el 87.5% de la duración total del bit.

Casos de Ajuste de Flanco y Resincronización (Nominal Bit Time)

En el protocolo CAN, la resincronización continua de los nodos receptores se realiza exclusivamente sobre los flancos descendentes ($1 \rightarrow 0$).

Caso 1: Flanco Ideal (Sin Ajuste)



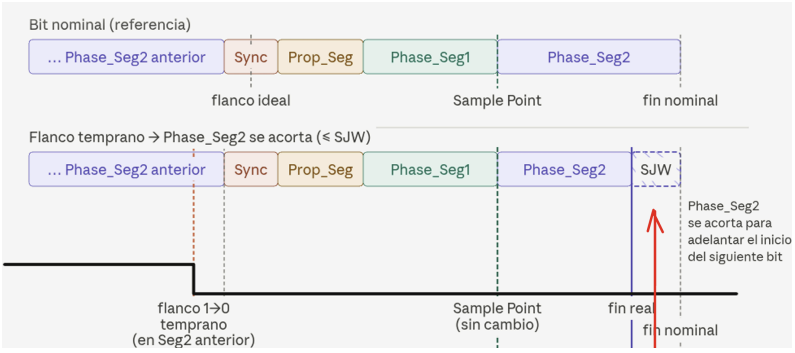
- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega exactamente dentro del segmento `Sync_Seg` (de duración fija $1t_q$).
- **Estado de Reloj:** El oscilador del nodo receptor está perfectamente sincronizado con el del transmisor.
- **Acción:** No se realiza ningún ajuste en la duración de los segmentos ($\Delta t_q = 0$) y el **Sample Point** permanece en su posición nominal.

Caso 2: Flanco Tardío (Reloj del Transmisor Más Lento)



- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega con retraso, cayendo en los segmentos `Prop_Seg` o `Phase_Seg1` en lugar de `Sync_Seg`.
- **Estado de Reloj:** El reloj del nodo transmisor es **más lento** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **alarga** el segmento `Phase_Seg1` agregando ciclos t_q adicionales.
  - La cantidad de ciclos agregados es igual al error de fase detectado, acotado por el valor máximo de `SJW` ($+\Delta t_q \leq \text{SJW}$).
  - **Efecto:** Se desplaza el **Sample Point** hacia la derecha (retrasando el momento de muestreo) para capturar correctamente el dato.

Caso 3: Flanco Temprano (Reloj del Transmisor Más Rápido)



- **Ubicación del Flanco:** El flanco descendente ($1 \to 0$) llega antes de lo esperado, cayendo dentro del segmento `Phase_Seg2` del bit anterior.
- **Estado de Reloj:** El reloj del nodo transmisor es **más rápido** que el del nodo receptor.
- **Mecanismo de Corrección:**
  - Se **acorta** el segmento `Phase_Seg2` substrayendo ciclos t_q de su duración.
  - La reducción está acotada por el valor máximo configurado en `SJW` ($-\Delta t_q \leq \text{SJW}$).
  - **Efecto:** Adelanta la finalización del bit actual e inicia el próximo ciclo antes de lo nominal para reengancharse con el transmisor rápido.

Caso	Ubicación del Flanco	Clock Transmisor	Ajuste de Hardware
Caso 1: Ideal	Dentro de `Sync_Seg`	Sincronizado	Sin ajuste ($\Delta t_q = 0$).
Caso 2: Tardío	En `Prop_Seg` / `Phase_Seg1`	Más lento	Se alarga `Phase_Seg1` ($+\Delta t_q \leq \text{SJW}$).
Caso 3: Temprano	En `Phase_Seg2` anterior	Más rápido	Se acorta `Phase_Seg2` ($-\Delta t_q \leq \text{SJW}$).

7.1.5.1 Resincronización Dinámica y SJW

Para absorber el **jitter** y las diferencias de frecuencia entre los osciladores a cristal de los distintos nodos, CAN realiza resincronizaciones continuas sobre los flancos de la señal.

> ⚙️ **Synchronization Jump Width (SJW)**
>
> El parámetro **SJW** (de 1 a $4t_q$) define la cantidad máxima de tiempo que `Phase_Seg1` puede **alargarse** o `Phase_Seg2` puede **acortarse** durante un evento de resincronización.
>
> - **Flanco Tardío (Reloj del transmisor más lento):** El flanco $1 \to 0$ cae dentro de `Prop_Seg` o `Phase_Seg1`. El nodo alarga `Phase_Seg1` hasta en $+\ \text{SJW}\ t_q$ para desplazar el **Sample Point** a la derecha.
> - **Flanco Temprano (Reloj del transmisor más rápido):** El flanco $1 \to 0$ cae dentro de `Phase_Seg2` del bit anterior. El nodo acorta `Phase_Seg2` en hasta $-\ \text{SJW}\ t_q$ para adelantar el inicio del próximo bit.

> ⚠️ **¡Sincronización Solo en Flancos Descendentes!**
>
> En CAN, la sincronización se realiza **exclusivamente sobre los flancos descendentes ($1 \to 0$, paso a dominante)**.
>
> Los flancos ascendentes ($0 \to 1$, paso a recesivo) **no se utilizan** debido a que la subida del bus es pasiva (depende de la resistencia **pull-middle** y capacitancias parásitas del cable), generando pendientes lentas e imprecisas.

Filtrado por Hardware y Manejo de Errores

7.1.6.1 Filtrado de Mensajes (Filter y Mask)

Para evitar sobrecargar la CPU del microcontrolador analizando mensajes no relevantes, el módulo controlador CAN (como el FlexCAN en el Kinetis K64F) filtra las tramas por hardware mediante dos registros:

- **Filter (Identificador de Filtro):** Contiene el patrón binario de ID específico que el nodo está programado para recibir.
- **Mask (Máscara de Aceptación):** Define qué bits del ID deben coincidir obligatoriamente:
  - Bit de máscara en `'1'`: **Verificación estricta** (el bit del ID entrante debe ser idéntico al bit del filtro).
  - Bit de máscara en `'0'`: **Don't Care** (se ignora la coincidencia de ese bit en particular).

7.1.6.2 Estados del Nodo y Manejo de Errores

Cada nodo CAN posee dos contadores internos de error mantenidos por hardware: `TEC` (**Transmit Error Counter**) y `REC` (**Receive Error Counter**). Según sus valores, el nodo transiciona entre tres estados de operación:

Estado del Nodo	Condición de Errores	Comportamiento en la Red
Error Active	$\text{TEC} < 128$ y $\text{REC} < 128$	Operación normal. Al detectar un error, transmite un **Active Error Flag** (6 bits dominantes), abortando la trama de toda la red.

Estado del Nodo	Condición de Errores	Comportamiento en la Red
Error Passive	$\mathrm{TEC} \geq 128$ o $\mathrm{REC} \geq 128$	El nodo está degradado. Transmite un **Passive Error Flag** (6 bits recesivos), evitando interrumpir la comunicación de otros nodos.
Bus Off	$\mathrm{TEC} > 255$	El nodo se desconecta lógicamente del bus. No puede transmitir ni recibir hasta reiniciar el módulo FlexCAN.